ALGORITHMS AND DATA STRUCTURES I

Rigorous Lecture Notes

A complete theory book for the first-semester examination

RAM model, sorting, selection, linear structures, heaps, range queries, balanced search trees, hashing, filters, and randomized structures

Format:	monochrome mathematical lecture notes
Coverage:	all 80 items of the supplied syllabus
Prepared for:	Timur Saar
Edition:	July 2026

Preface

These notes develop the complete theory required by the supplied first-semester examination programme in Algorithms and Data Structures I. The goal is not to memorize eighty isolated answers. It is to learn a small collection of proof methods—loop invariants, induction on a recursive call tree, exchange arguments, potential functions, and probabilistic counting—and to see how they generate the entire syllabus.

Every algorithmic section follows the same contract.

1. The computational problem and all preconditions are stated precisely.
2. A representation invariant is separated from the implementation.
3. The algorithm is given in conventional pseudocode or C++-style notation.
4. Correctness is proved, including termination where it is not immediate.
5. Running time and memory usage are derived in the declared model.

The source programme marks a few sophisticated results as usable without proof. This book goes further: it proves the central result whenever a self-contained proof is appropriate for a first course, and gives a rigorous proof skeleton with its exact assumptions for the remaining advanced probabilistic bounds. Thus no asymptotic claim rests on an unnamed intuition.

How the syllabus is encoded

Each relevant section begins with a framed label such as EXAMINATION SYLLABUS: ITEM 26. The labels may be read as a checklist. Appendix A maps all items $1, \dots, 80$ to their locations. Closely related items are sometimes treated together, but none is omitted.

Mathematical conventions

Unless a section says otherwise:

- arrays are indexed from 0 through $n - 1$;
- ranges $[l, r)$ are half-open and have length $r - l$;
- logarithms have base 2, while ln denotes the natural logarithm;
- a comparison model exposes keys only through comparisons;
- randomized bounds are expectations over the algorithm's internal random choices, with the input held fixed;
- $\mathcal{O}(1)$-time arithmetic is asserted only for values fitting in one RAM word, whose size is at least $\lceil \log_2(n+1) \rceil$ bits.

For a randomized running time T, "expected $\mathcal{O}(f(n))$" means

$$\sup_{x:\,|x|=n} \mathbb{E}[T(x)] = \mathcal{O}(f(n)),$$

unless an average-case distribution on inputs is explicitly specified.

The proof toolkit

Five templates recur throughout the book.

Invariant proof.
State a predicate before the loop; prove initialization, preservation, and that the predicate plus loop termination implies the postcondition.

Structural induction.
Prove a claim for an empty structure, then assume it for recursive children and derive it for the parent. This is the natural language of trees.

Charging argument.
Associate expensive work with objects or events, and prove that each can be charged only a bounded number of times.

Decision-tree argument.
Represent every possible comparison transcript as a root-to-leaf path; count how many leaves correctness requires.

Random indicator argument.
Write a count as $X = \sum_i X_i$ with $X_i \in \{0, 1\}$, then use $\mathbb{E}[X] = \sum_i \mathbb{P}(X_i = 1)$. Independence is not needed for linearity of expectation.

Prerequisites and scope

The text assumes elementary discrete mathematics: induction, finite sums, logarithms, basic probability, and elementary modular arithmetic. It develops all data-structure-specific results from definitions. Machine engineering details such as cache-line sizes are mentioned only when they explain the practical role of a structure; the formal complexity results are proved in the RAM, comparison, or external-memory model stated at the point of use.

Suggested examination method

For each syllabus label, practise giving a five-part oral answer: problem, invariant, algorithm, correctness, complexity. A convincing proof normally begins with the invariant rather than with code. If a data structure supports several operations, first state its representation invariant once, then prove each operation preserves it. This organization is both shorter and more rigorous.

Contents

Part I

Foundations, Arrays, Sorting, and Selection

LECTURE UNIT 1

Computation, Growth, and Static Arrays

1.1 The word-RAM model

EXAMINATION SYLLABUS: ITEM 1

An algorithm is not assigned a running time until a machine model specifies which elementary operations cost one step. The standard model for this course is the **word random-access machine**, or word-RAM.

Definition 1.1 (Word-RAM)**.** For inputs of size n, a word-RAM has an unbounded sequence of memory words, each containing $w \geq \lceil \log_2(n+1) \rceil$ bits. Reading or writing a word at an address contained in a word costs one step. The following operations on a constant number of words also cost one step:

$$+, \quad -, \quad \times, \quad \lfloor x/y \rfloor, \quad x \bmod y,$$
$$<, \quad =, \quad \wedge, \quad \vee, \quad \oplus, \quad \neg, \quad \text{fixed-distance shifts.}$$

The result of a unit-cost operation must fit in $\mathcal{O}(1)$ words. Input and output cost one step per accessed word.

The condition $w \geq \log_2(n+1)$ allows any array index to fit in one word. Often we additionally assume $w = \Theta(\log n)$, which prevents an algorithm from hiding an exponentially large integer in one constant-time word. Arbitrary-length integer arithmetic is therefore *not* automatically constant time.

Remark 1.2 (Uniformity). The programme executed by a RAM is finite and independent of n. One may not place the answer to every input of length n in the instruction set. This uniformity condition is usually implicit.

1.1.1 Worst-case, expected, and amortized time

For a deterministic algorithm A, let $T_A(x)$ be the number of RAM steps on input x. Its worst-case time on inputs of size n is

$$T_A(n) = \max_{|x|=n} T_A(x).$$

If A is randomized, $T_A(x, \omega)$ also depends on random coins ω. The expected worst-input time is

$$T_A(n) = \max_{|x|=n} \mathbb{E}_\omega[T_A(x, \omega)].$$

Amortized analysis, developed in section 5.9, is different: it bounds the total cost of every operation sequence, without any probability distribution.

1.1.2 Counting operations: a first example

Consider a linear scan that returns the first occurrence of x.

Listing 1.1: Linear search

```
function LinearSearch(a[0..n-1], x):
    for i = 0 to n - 1:
        if a[i] == x:
            return i
    return NOT_FOUND
```

Each iteration performs a constant number of word operations. On an array not containing x, all n iterations execute, so $T(n) \le c_1 n + c_0$ for some machine-dependent constants. Conversely, merely reading the n distinct array positions costs at least n steps on this execution, so $T(n) \ge n$. Hence $T(n) = \Theta(n)$. Asymptotic notation deliberately suppresses the constants c_0, c_1, but not the dependence on n.

Common pitfall 1.3. Writing "there is one loop, therefore the time is $\mathcal{O}(n)$" is not a proof. The body might itself take nonconstant time, the loop variable might double, or the execution might terminate early. Count iterations and bound the cost of one iteration separately.

1.2 Asymptotic notation

EXAMINATION SYLLABUS: ITEM 2

Let $f, g : \mathbb{N} \to \mathbb{R}_{\ge 0}$ from some index onward. Eventual nonnegativity avoids irrelevant sign complications in running-time functions.

Definition 1.4 (Big O)**.** $f(n) = \mathcal{O}(g(n))$ if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$0 \le f(n) \le cg(n) \qquad \text{for every } n \ge n_0.$$

Definition 1.5 (Big Omega and Theta)**.** $f(n) = \Omega(g(n))$ if there exist $c > 0, n_0$ such that $0 \le cg(n) \le f(n)$ for all $n \ge n_0$. Moreover,

$$f(n) = \Theta(g(n)) \quad \iff \quad f(n) = \mathcal{O}(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

The equality sign is conventional but asymmetric: $2n = \mathcal{O}(n^2)$ is true, whereas the expression $\mathcal{O}(n^2) = 2n$ should not be used. A safer reading is $f \in \mathcal{O}(g)$, where $\mathcal{O}(g)$ is a set of functions.

1.2.1 Independence of the starting index

Sequences are sometimes indexed by $0, 1, \ldots$ and sometimes by $1, 2, \ldots$. The asymptotic class does not depend on any finite prefix.

Theorem 1.6 (Finite-prefix invariance)**.** *Suppose f, g and $\widetilde{f}, \widetilde{g}$ are eventually nonnegative and there exists N such that*

$$f(n) = \widetilde{f}(n), \qquad g(n) = \widetilde{g}(n) \quad \text{for every } n \geq N.$$

Then $f = \mathcal{O}(g)$ if and only if $\widetilde{f} = \mathcal{O}(\widetilde{g})$. The analogous statements hold for Ω and Θ.

Proof. Assume $f = \mathcal{O}(g)$, witnessed by $c > 0, n_0$. For every $n \geq \max\{N, n_0\}$,

$$\widetilde{f}(n) = f(n) \leq cg(n) = c\widetilde{g}(n).$$

Thus the same c and the new threshold $\max\{N, n_0\}$ witness $\widetilde{f} = \mathcal{O}(\widetilde{g})$. Interchanging the tilded and untilded functions proves the converse. Big Omega follows by reversing the inequality; Theta follows by conjunction. □

A shift of the argument also changes nothing for standard monotone growth rates.

Proposition 1.7 (Fixed shifts)**.** *For every fixed integer a and every $d \geq 0$,*

$$(n + a)^d = \Theta(n^d), \qquad \log(n + a) = \Theta(\log n)$$

on indices where the expressions are defined.

Proof. For $n \geq 2|a| + 1$, $n/2 \leq n + a \leq 3n/2$. Raising to the fixed power d gives constant-factor upper and lower bounds. Taking logarithms gives $\log n - 1 \leq \log(n + a) \leq \log n + \log(3/2)$; for $n \geq 2$, the additive constants are absorbed by constant factors. □

1.2.2 Calculation rules

Proposition 1.8 (Algebra of asymptotic bounds)**.** *For eventually nonnegative functions:*

1. *if $f = \mathcal{O}(g)$ and $g = \mathcal{O}(h)$, then $f = \mathcal{O}(h)$;*
2. *if $f_1 = \mathcal{O}(g_1)$ and $f_2 = \mathcal{O}(g_2)$, then $f_1 + f_2 = \mathcal{O}(g_1 + g_2)$;*
3. *under the same assumptions, $f_1 f_2 = \mathcal{O}(g_1 g_2)$;*
4. *if $f = \mathcal{O}(g)$, then $af = \mathcal{O}(g)$ for every fixed $a > 0$;*
5. *$f + g = \Theta(\max\{f, g\})$.*

Proof. For 1, multiply the two witnessing constants and take the larger threshold. For 2, beyond the larger threshold, $f_1 + f_2 \leq c_1 g_1 + c_2 g_2 \leq \max(c_1, c_2)(g_1 + g_2)$. For 3, multiply the inequalities. Statement 4 is immediate. Finally,

$$\max\{f, g\} \leq f + g \leq 2 \max\{f, g\},$$

which proves 5. □

1.2.3 Examples and strict distinctions

For fixed constants $a > 1$, $b > 1$, $k \geq 0$, and $\varepsilon > 0$,

$$1 \ll \log^k n \ll n^\varepsilon \ll a^n \ll n! \ll n^n,$$

where $f \ll g$ means $f = o(g)$. Two particularly useful limits are

$$\lim_{n\to\infty} \frac{\log^k n}{n^\varepsilon} = 0, \qquad \lim_{n\to\infty} \frac{n^k}{a^n} = 0.$$

They follow, for example, by repeated application of l'Hopital's rule to the corresponding real functions, or by comparing ratios of consecutive terms.

Example 1.9. $3n^2 + 17n + 4 = \Theta(n^2)$: for $n \geq 1$, the expression is at least $3n^2$ and at most $24n^2$. In contrast, $n^2 \notin \mathcal{O}(n \log n)$, because $n^2/(n \log n) = n/\log n \to \infty$.

Example 1.10 (A sum)**.** The loop whose i-th iteration performs i constant-time operations has cost

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

A nested loop is therefore not automatically quadratic: its bounds determine the sum that must be evaluated.

1.3 Static range sums by prefix sums

EXAMINATION SYLLABUS: ITEM 3

Problem. Given a fixed array $a[0], \ldots, a[n-1]$ over an additive group, preprocess it so that each query

$$\text{RangeSum}(l, r) = \sum_{i=l}^{r-1} a[i], \qquad 0 \leq l \leq r \leq n,$$

is answered quickly. "Static" means no element changes after preprocessing.

Definition 1.11 (Prefix sums)**.** Define an array $p[0], \ldots, p[n]$ by

$$p[0] = 0, \qquad p[i+1] = p[i] + a[i].$$

Listing 1.2: Prefix-sum preprocessing and queries

```
vector<long long> buildPrefix(const vector<long long>& a) {
    vector<long long> p(a.size() + 1, 0);
    for (size_t i = 0; i < a.size(); ++i)
        p[i + 1] = p[i] + a[i];
    return p;
}

long long rangeSum(const vector<long long>& p, int l, int r) {
    return p[r] - p[l];               // sum on [l, r)
}
```

Theorem 1.12 (Correctness of prefix sums)**.** *After preprocessing,*

$$p[j] = \sum_{i=0}^{j-1} a[i] \quad (0 \le j \le n),$$

and consequently $p[r] - p[l] = \sum_{i=l}^{r-1} a[i]$.

Proof. Induct on j. The claim for $j = 0$ is the empty sum. If it holds for j, then

$$p[j+1] = p[j] + a[j] = \sum_{i=0}^{j-1} a[i] + a[j] = \sum_{i=0}^{j} a[i].$$

For a query, split the prefix at l:

$$p[r] = \sum_{i=0}^{l-1} a[i] + \sum_{i=l}^{r-1} a[i] = p[l] + \sum_{i=l}^{r-1} a[i].$$

Subtracting $p[l]$ proves the formula. This uses additive inverses; for a monoid without inverses, a general range query cannot necessarily be obtained from two prefix aggregates. □

Preprocessing uses n additions and $n+1$ words, hence $\Theta(n)$ time and $\Theta(n)$ additional memory. A query uses two array accesses and one subtraction, hence $\Theta(1)$ time.

Common pitfall 1.13 (Overflow and range convention). If $a[i]$ is a 32-bit integer, a prefix sum may require 64 bits. Also decide once whether queries use $[l, r)$ or $[l, r]$. With half-open ranges, the empty range is represented naturally and the formula has no $l - 1$ boundary case.

1.4 Binary search in a sorted array

Examination syllabus: item 4

Problem. Given a nondecreasing array $a[0..n-1]$ and a key x, determine whether x occurs. A stronger primitive is

$$\text{lower\_bound}(x) = \min\{i \in [0, n] : i = n \text{ or } a[i] \geq x\}.$$

Membership then holds exactly when the returned index is less than n and its element equals x.

Listing 1.3: Lower bound on a half-open search interval

```
int lowerBound(const vector<int>& a, int x) {
    int l = 0;
    int r = static_cast<int>(a.size());
    while (l < r) {
        int m = l + (r - l) / 2;       // avoids l + r overflow
        if (a[m] < x)
            l = m + 1;
        else
            r = m;
    }
    return l;
}
```

Invariant 1.14 (Binary-search invariant)**.** At the beginning of every iteration:

$$0 \leq l \leq r \leq n, \qquad a[i] < x \text{ for all } i < l, \qquad a[i] \geq x \text{ for all } i \geq r.$$

The middle interval $[l, r)$ contains every still-possible answer.

Theorem 1.15 (Correctness of lower bound)**.** *The algorithm terminates and returns the least index $i \in [0, n]$ such that $i = n$ or $a[i] \geq x$.*

Proof. Initially $l = 0, r = n$. The assertions about indices below l and at least r are vacuous, so the invariant holds.

Let $m = l + \lfloor (r - l)/2 \rfloor$. If $a[m] < x$, sortedness implies $a[i] \leq a[m] < x$ for every $i \leq m$. Setting $l = m + 1$ therefore preserves the left assertion, and the right assertion is unchanged. Otherwise $a[m] \geq x$; sortedness implies $a[i] \geq x$ for every $i \geq m$. Setting $r = m$ preserves the right assertion, while the left one is unchanged.

Whenever $l < r$, the new interval is strictly shorter: its length is at most $\lceil (r - l)/2 \rceil$. Thus termination is guaranteed. At termination $l = r$. The invariant says all indices below l contain values less than x, and all indices at least l, if any, contain values at least x. Hence l is the required least index. □

After t iterations the candidate interval has length at most $\lceil n/2^t \rceil$. It becomes empty when $2^t > n$, so the time is $\mathcal{O}(\log n)$; for instances whose answer cannot be known before distinguishing among $n+1$ positions, a binary decision tree needs height at least $\lceil \log_2(n+1) \rceil$, giving $\Theta(\log n)$ comparisons in the worst case. Iterative binary search uses $\Theta(1)$ additional memory.

Remark 1.16 (Other boundary searches). Replacing $a[m] < x$ with $a[m] \leq x$ produces `upperBound`, the first index with value $> x$. Consequently the number of occurrences of x is

$$\text{upper\_bound}(x) - \text{lower\_bound}(x).$$

LECTURE UNIT 2

Comparison Sorting and Divide-and-Conquer

2.1 The sorting problem and two equivalent formulations

EXAMINATION SYLLABUS: ITEM 5

Let $A = (a_0, \ldots, a_{n-1})$ be records whose keys belong to a totally ordered set. Equal keys are permitted.

Definition 2.1 (Value-sorting formulation)**.** Produce a sequence $B = (b_0, \ldots, b_{n-1})$ such that

1. $b_0 \le b_1 \le \cdots \le b_{n-1}$, and
2. B is a permutation of A, counting multiplicities.

Definition 2.2 (Index-sorting or argsort formulation)**.** Produce a permutation π of $\{0, \ldots, n-1\}$ such that

$$a_{\pi(0)} \le a_{\pi(1)} \le \cdots \le a_{\pi(n-1)}.$$

A stable argsort additionally requires $\pi(i) < \pi(j)$ whenever the two selected keys are equal and $i < j$ in the output order.

Theorem 2.3 (Equivalence of the formulations)**.** *In the RAM comparison model, either formulation can be reduced to the other with $\mathcal{O}(n)$ additional work and $\mathcal{O}(n)$ words of output storage. The number of key comparisons changes by at most $\mathcal{O}(n)$.*

Proof. Given an argsort permutation π, set $b_i = a_{\pi(i)}$ for every i; this takes n reads and writes.

Conversely, attach its original index to each record and run the value sorter on pairs

$$(a_i, i), \qquad (a_i, i) < (a_j, j) Longleftrightarrow a_i < a_j \text{ or } (a_i = a_j \text{ and } i < j).$$

The second components of the sorted pairs form π. Each pair comparison uses one key comparison, with a constant amount of index work, and constructing and projecting the pairs takes $\mathcal{O}(n)$ time. The tie-breaking also yields stability. □

This equivalence formalizes why a sorting algorithm may be described as moving values even when an implementation moves records or pointers. A correct sorting proof must establish both order and preservation of the multiset; order alone could be achieved by outputting n copies of the minimum.

2.2 The growth of $\log(n!)$

EXAMINATION SYLLABUS: ITEM 6

The factorial appears because n distinct items have $n!$ possible input orders.

Theorem 2.4. *For $n \geq 2$,*

$$\log(n!) = \Theta(n \log n).$$

Proof. The upper bound follows from $i \leq n$ for every $1 \leq i \leq n$:

$$n! = \prod_{i=1}^{n} i \leq n^n, \qquad \log(n!) \leq n \log n.$$

For the lower bound, at least $\lceil n/2 \rceil$ factors in $n!$ are at least $\lfloor n/2 \rfloor$. For $n \geq 4$,

$$n! \geq \left(\frac{n}{2}\right)^{n/2}, \qquad \log(n!) \geq \frac{n}{2} \log \frac{n}{2} = \frac{n}{2}(\log n - 1) \geq \frac{n}{4} \log n.$$

Changing the bound on finitely many smaller n does not affect asymptotics. Together the inequalities prove the claim. □

For sharper intuition, Stirling's formula gives

$$\log_2(n!) = n \log_2 n - (\log_2 e)n + \mathcal{O}(\log n),$$

but the syllabus result does not require Stirling's formula.

2.3 A lower bound for comparison sorting

EXAMINATION SYLLABUS: ITEM 7

In a **comparison sorting algorithm**, information about keys is obtained only through questions such as $a_i < a_j$. Arithmetic on keys, table indexing by key value, and digit inspection are forbidden. Algorithms such as counting sort do not belong to this model.

Definition 2.5 (Comparison decision tree)**.** For a fixed n, an execution of a deterministic comparison algorithm is represented by a rooted tree. Each internal node is a comparison; its outgoing edges are possible results; and each leaf specifies the output permutation. A root-to-leaf length is the number of comparisons on that execution.

For distinct keys, every comparison has two possible outcomes. It is therefore enough to consider binary trees.

Theorem 2.6 (Comparison-sorting lower bound)**.** *Every deterministic comparison algorithm that correctly sorts n distinct keys performs at least*

$$\lceil \log_2(n!) \rceil = \Omega(n \log n)$$

comparisons on some input. The same asymptotic lower bound holds for the expected number of comparisons of every randomized comparison sorter on some input.

Deterministic case. Fix n distinct labeled items. There are $n!$ possible total orders of their keys. Two different total orders cannot reach the same leaf: the leaf prescribes one output ordering and would be wrong for at least one of them. Thus the decision tree has at least $n!$ leaves.

A binary tree of height h has at most 2^h leaves. Hence

$$2^h \geq n!, \qquad h \geq \lceil \log_2(n!) \rceil = \Omega(n \log n),$$

using the preceding theorem. Height is the maximum number of comparisons. □

Randomized case. Consider the uniform distribution over the $n!$ input orders. Fixing all random coins turns the randomized algorithm into a deterministic decision tree. In any binary tree with L leaves, the average depth of L distinct leaves is at least $\log_2 L$. Indeed, the leaf depths d_i satisfy Kraft's inequality $\sum_i 2^{-d_i} \leq 1$; since 2^{-x} is convex, Jensen's inequality gives $2^{-\frac{1}{L}\sum d_i} \leq 1/L$, hence $L^{-1} \sum d_i \geq \log_2 L$.

Therefore, for every fixing of the coins, the mean cost over uniform inputs is at least $\log_2(n!)$. Averaging also over the coins preserves this lower bound. Consequently at least one input has expected cost at least the average. This is the elementary form of Yao's minimax principle needed here. □

The lower bound counts comparisons, not RAM instructions, and applies to the worst case. It does not prevent linear-time sorting when keys lie in a suitably small integer universe.

2.4 Merge sort

EXAMINATION SYLLABUS: ITEM 8

Merge sort divides the array into two halves, recursively sorts them, and merges the two sorted sequences.

Listing 2.1: Stable merge sort on a half-open range

```
1 procedure MergeSort(a, l, r):
2     if r - l <= 1:
3         return
4     m = l + floor((r - l) / 2)
5     MergeSort(a, l, m)
6     MergeSort(a, m, r)
7     Merge(a, l, m, r)
8
```

```
9  procedure Merge(a, l, m, r):
10     i = l; j = m; temporary = empty array
11     while i < m and j < r:
12         if a[i] <= a[j]:              // <= makes the merge stable
13             temporary.push_back(a[i]); i = i + 1
14         else:
15             temporary.push_back(a[j]); j = j + 1
16     append a[i..m) to temporary
17     append a[j..r) to temporary
18     copy temporary back to a[l..r)
```

Lemma 2.7 (Correctness of merge)**.** *If $a[l..m)$ and $a[m..r)$ are nondecreasing before* `Merge`*, then after the procedure $a[l..r)$ is a stable sorted permutation of their union.*

Proof. At the beginning of each main-loop iteration, the temporary array is a stable sorted sequence consisting precisely of the already-consumed elements, and every unconsumed element is at least its own half's current first element. The smaller of the two current first elements is therefore the smallest unconsumed element globally. Appending it preserves sortedness and multiset equality. On equality, choosing the left copy preserves original relative order.

When one half is exhausted, the other half is already sorted and every one of its elements is at least the last emitted element, so appending it preserves the invariant. Every element is emitted exactly once. Copying changes only location, not order or multiplicity. □

Theorem 2.8 (Correctness of merge sort)**.** *For every $0 \le l \le r \le n$,* `MergeSort(a,l,r)` *terminates and replaces $a[l..r)$ by a stable nondecreasing permutation of its original elements.*

Proof. Use strong induction on $s = r - l$. If $s \le 1$, the range is already sorted. For $s \ge 2$, both recursive ranges have size strictly below s, so they terminate and are sorted permutations by induction. The merge lemma then proves the claim for the entire range. Recursion terminates because sizes strictly decrease to the base case. □

Merging s elements takes at most $s - 1$ key comparisons and $\Theta(s)$ RAM steps. Thus merge sort has recurrence

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \log n).$$

The conventional implementation uses $\Theta(n)$ auxiliary array storage and $\Theta(\log n)$ recursion depth. It is stable because the left element wins a tie.

2.5 Solving $T(n) = 2T(n/2) + \mathcal{O}(n)$

EXAMINATION SYLLABUS: ITEM 9

The notation $T(n) = 2T(n/2) + \mathcal{O}(n)$ is shorthand for an inequality: there are constants c, n_0 such that

$$T(n) \le 2T(n/2) + cn$$

for sufficiently large n, usually first considered for powers of two.

Theorem 2.9 (Upper-bound recurrence)**.** *If $T(1) \le d$ and $T(n) \le 2T(n/2) + cn$ for powers of two $n \ge 2$, then*

$$T(n) \le dn + cn \log_2 n = \mathcal{O}(n \log n).$$

Recursion-tree proof. At level j there are 2^j subproblems of size $n/2^j$. Their total nonrecursive cost is at most

$$2^j c \frac{n}{2^j} = cn.$$

There are $\log_2 n$ non-leaf levels, contributing at most $cn \log_2 n$. The n leaves contribute at most dn. □

Induction proof. The formula is true at $n = 1$. Assuming it at $n/2$,

$$\begin{aligned} T(n) &\le 2\left(d\frac{n}{2} + c\frac{n}{2}\log_2 \frac{n}{2}\right) + cn \\ &= dn + cn(\log_2 n - 1) + cn = dn + cn \log_2 n. \end{aligned}$$

□

If the recurrence also has a matching lower toll cn, namely $T(n) \ge 2T(n/2) + c'n$ with $c' > 0$, the same level sum gives $T(n) = \Theta(n \log n)$. Floors and ceilings change only constants: one may pad to the next power of two $N < 2n$, or carry unequal halves through the induction.

2.6 Counting inversions

EXAMINATION SYLLABUS: ITEM 10

Definition 2.10 (Inversion)**.** An inversion of $a[0..n-1]$ is a pair (i, j) such that

$$0 \le i < j < n \quad \text{and} \quad a[i] > a[j].$$

The direct double loop takes $\Theta(n^2)$ time. Merge sort counts inversions in $\Theta(n \log n)$ by separating them into left, right, and crossing inversions.

Listing 2.2: Merge while counting crossing inversions

```
function SortAndCount(a, l, r):
    if r - l <= 1: return 0
    m = l + floor((r - l) / 2)
    answer = SortAndCount(a, l, m) + SortAndCount(a, m, r)
    i = l; j = m; temporary = empty array
    while i < m and j < r:
        if a[i] <= a[j]:
```

```
            temporary.push_back(a[i]); i = i + 1
        else:
            temporary.push_back(a[j]); j = j + 1
            answer = answer + (m - i)
    append the remainders and copy temporary to a[l..r)
    return answer
```

Theorem 2.11 (Correctness of inversion counting)**.** *`SortAndCount(a,l,r)` returns the number of inversions whose two indices lie in $[l, r)$, and sorts that range.*

Proof. Induct on the range length. Recursive calls correctly count inversions contained entirely in one half. It remains to count crossing pairs $i < m \le j$.

During merge, if the left current value is at most the right current value, it forms no inversion with that right value or any later (no smaller) right value. If $a[j] < a[i]$, then because the left half is sorted,

$$a[j] < a[i] \le a[i+1] \le \cdots \le a[m-1].$$

Thus the selected right element forms exactly $m - i$ crossing inversions with all unconsumed left elements. Each crossing inversion is counted precisely when its right endpoint is emitted before its left endpoint. The three classes of inversions are disjoint and exhaustive. Merge correctness also sorts the range. □

The recurrence is the merge-sort recurrence, so time is $\Theta(n \log n)$ and auxiliary memory is $\Theta(n)$. Use a sufficiently wide integer: the maximum number of inversions is $n(n-1)/2$.

2.7 Randomized quicksort

EXAMINATION SYLLABUS: ITEM 11

Quicksort chooses a pivot, partitions elements into those below, equal to, and above it, and recursively sorts the outer parts. A three-way partition prevents many equal keys from causing quadratic recursion.

Listing 2.3: Randomized three-way quicksort

```
procedure QuickSort(a, l, r):
    if r - l <= 1: return
    p = a[random integer in [l, r)]
    lt = l; i = l; gt = r
    // a[l..lt) < p, a[lt..i) == p, a[gt..r) > p
    while i < gt:
        if a[i] < p:
            swap(a[lt], a[i]); lt = lt + 1; i = i + 1
        else if p < a[i]:
            gt = gt - 1; swap(a[i], a[gt])
        else:
            i = i + 1
```

```
13      QuickSort(a, l, lt)
14      QuickSort(a, gt, r)
```

Lemma 2.12 (Partition correctness)**.** *At termination, a[l..lt<p), a[lt..gt=p), and a[gt..r>p), and the range contains exactly its original multiset.*

Proof. The comment in the algorithm is a loop invariant; additionally, $a[i..gt)$ is the unclassified region. Each branch moves the current element to its correct classified region and shrinks the unclassified region by one. Swaps preserve the multiset. When $i = gt$, no unclassified elements remain. □

Theorem 2.13 (Quicksort correctness)**.** *Randomized quicksort terminates and sorts the input range for every sequence of random choices.*

Proof. Induct on range length. Partition is correct. Every pivot-equal element lies in the middle block, which is nonempty, so each recursive range is strictly shorter. By induction they become sorted. Since every left key is below every middle key, and every middle key below every right key, their concatenation is sorted. □

An adversarial sequence of extreme pivots gives $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Random pivot selection removes dependence on the original input order.

Theorem 2.14 (Expected quicksort comparisons)**.** *On n distinct keys, randomized quicksort performs at most*

$$2(n+1)H_n - 4n = \mathcal{O}(n \log n)$$

expected key comparisons, where $H_n = \sum_{k=1}^{n} 1/k$.

Proof. Name the sorted keys $z_1 < \cdots < z_n$. A pair z_i, z_j, $i < j$, is compared exactly when the first pivot chosen from $\{z_i, z_{i+1}, \ldots, z_j\}$ is one of its two endpoints. If an interior key is chosen first, it separates the pair forever; if an endpoint is chosen first, the partition compares it with the other endpoint. Uniform pivot choices imply

$$\mathbb{P}(z_i \text{ compared with } z_j) = \frac{2}{j-i+1}.$$

Let X_{ij} be the indicator of this comparison. By linearity of expectation,

$$\begin{aligned}\mathbb{E}[C_n] &= \sum_{1 \le i < j \le n} \frac{2}{j-i+1} \\ &= 2 \sum_{d=1}^{n-1} \frac{n-d}{d+1} = 2(n+1)H_n - 4n.\end{aligned}$$

Since $H_n \le 1 + \ln n$, this is $\mathcal{O}(n \log n)$. Partition overhead is linear in the number of comparisons, so expected RAM time has the same bound. □

The algorithm sorts in place apart from recursion. Random recursion depth is $\mathcal{O}(\log n)$ in expectation but can be n. To guarantee $\mathcal{O}(\log n)$ stack words, recurse on the smaller side and process the larger side in a loop; the recursively chosen side then has at most half the current length.

LECTURE UNIT 3

Selection and Integer Sorting

3.1 Randomized quickselect

EXAMINATION SYLLABUS: ITEM 12

Problem. Given n keys and an integer $k \in \{0, \dots, n-1\}$, return the key of rank k: the element that would occupy position k in sorted order. This is the k-th zero-based **order statistic**. Sorting first costs $\Theta(n \log n)$; selection needs only expected linear time.

Listing 3.1: Quickselect with a random pivot

```
function QuickSelect(a, l, r, k):       // l <= k < r
    if r - l == 1: return a[l]
    p = a[random integer in [l, r)]
    (lt, gt) = ThreeWayPartition(a, l, r, p)
    if k < lt: return QuickSelect(a, l, lt, k)
    if k >= gt: return QuickSelect(a, gt, r, k)
    return p
```

Here `ThreeWayPartition` is the Dutch-national-flag procedure from chapter 2; it produces keys $< p$ in $[l, lt)$, keys $= p$ in $[lt, gt)$, and keys $> p$ in $[gt, r)$.

Theorem 3.1 (Quickselect correctness)**.** *For every pivot sequence, `QuickSelect(a,l,r,k)` terminates and returns the key of rank $k - l$ within the original multiset $a[l..r)$.*

Proof. Partition preserves the multiset and places every key below p before every copy of p, and every key above p after them. Therefore, if $k < lt$, the desired occurrence belongs to the left block; if $k \geq gt$, it belongs to the right block; otherwise its value is p. A recursive block is strictly shorter because the nonempty pivot-equal block is discarded. Strong induction on the range length proves correctness and termination. □

Theorem 3.2 (Expected linear time)**.** *With a uniformly random pivot, quickselect uses $\mathcal{O}(n)$ expected comparisons on every fixed input of n keys.*

Proof. It suffices to analyze distinct keys; three-way partition only discards more keys when duplicates occur. Let S be the size of the recursive subproblem after a random pivot. If the target rank is k and the pivot rank is r, then

$$S = \begin{cases} n - r - 1, & r < k, \\ 0, & r = k, \\ r, & r > k. \end{cases}$$

This expectation is largest for a central k; bounding each possible recursive side by the larger side gives the convenient universal estimate

$$\mathbb{E}[S] \le \frac{1}{n}\sum_{r=0}^{n-1} \max\{r, n-r-1\} \le \frac{3n}{4}.$$

Let $T(n)$ be the maximum expected time, and let partition cost at most cn. Strongly assume $T(m) \le Cm$ for all $m < n$. Conditioning on S and using the induction hypothesis,

$$T(n) \le cn + \mathbb{E}[T(S)] \le cn + C\mathbb{E}[S] \le cn + \frac{3C}{4}n \le Cn$$

when $C \ge 4c$, after enlarging C for base cases. Thus $T(n) = \mathcal{O}(n)$. Reading an arbitrary unsorted input requires $\Omega(n)$ inspections in the worst case, so the expected bound is optimal up to a constant. □

Worst-case time remains $\Theta(n^2)$ if every pivot is extreme. Randomization guarantees expectation, not a deterministic per-execution bound.

3.2 Deterministic linear-time selection: median of medians

EXAMINATION SYLLABUS: ITEM 13

The BFPRT algorithm manufactures a pivot that is never too far from the true median.

Listing 3.2: Deterministic selection (BFPRT)

```
function DeterministicSelect(a, l, r, k):
    n = r - l
    if n <= 50:
        sort a[l..r) by insertion sort
        return a[k]

    // Move the median of each group of at most five to a prefix.
    groups = ceil(n / 5)
    for g = 0 to groups - 1:
        L = l + 5*g; R = min(L + 5, r)
        sort a[L..R) by insertion sort
        M = L + floor((R - L) / 2)
        swap(a[l + g], a[M])

    p = DeterministicSelect(a, l, l + groups,
                            l + floor(groups / 2))
    (lt, gt) = ThreeWayPartition(a, l, r, p)
    if k < lt: return DeterministicSelect(a, l, lt, k)
    if k >= gt: return DeterministicSelect(a, gt, r, k)
    return p
```

Sorting a group of five takes constant time. The recursive selection among the group medians returns their median p.

Lemma 3.3 (Pivot quality)**.** *Except for at most two incomplete or exceptional groups, at least* $3n/10 - 6$ *input elements are at most* p*, and at least* $3n/10 - 6$ *are at least* p*. Consequently the post-partition recursive side has size at most* $7n/10 + 6$*.*

Proof. At least half of the group medians are at least the median p of all medians. For every complete group whose median is at least p, its median and its two larger elements are at least p. Ignoring the group containing p, a possible incomplete group, and rounding losses leaves at least

$$3\left(\frac{1}{2}\lfloor n/5\rfloor - 2\right) \geq \frac{3n}{10} - 6$$

elements at least p. The symmetric argument gives the same number at most p. Partition discards one of these sets together with the pivot block, so no recursive side exceeds $n-(3n/10-6) = 7n/10+6$. □

Theorem 3.4 (BFPRT correctness and complexity)**.** *Deterministic selection returns the requested order statistic in worst-case* $\Theta(n)$ *time.*

Proof. Correctness is identical to quickselect's inductive proof because the only change is the method of choosing p. Termination follows from the pivot-quality lemma outside the constant-size base case.

For complexity, group sorting, median movement, and partition cost cn for a constant c. The median recursion has size at most $\lceil n/5\rceil$, and the selection recursion at most $7n/10 + 6$. Hence

$$T(n) \leq T(\lceil n/5\rceil) + T(7n/10 + 6) + cn.$$

Choose a fixed base threshold n_0 large enough that $\lceil n/5\rceil + 7n/10 + 6 \leq (19/20)n$ for $n \geq n_0$. Strongly assuming $T(m) \leq Cm$ below n,

$$T(n) \leq C\frac{19n}{20} + cn \leq Cn$$

for $C \geq 20c$; enlarge C to cover $n < n_0$. Therefore $T(n) = \mathcal{O}(n)$. The $\Omega(n)$ input-inspection lower bound gives $\Theta(n)$. □

The number five is not mystical. Groups of three fail to make the two recursive fractions sum to less than one; any suitable larger odd constant also works, with different constants.

3.3 Deterministic $\Theta(n \log n)$ quicksort

EXAMINATION SYLLABUS: ITEM 14

Replace quicksort's random pivot by the exact median, found by BFPRT, and partition around it. For even n, choose either middle order statistic. The two recursive subarrays then have sizes at most $\lfloor n/2\rfloor$.

Listing 3.3: Worst-case optimal deterministic quicksort

```
1 procedure MedianQuickSort(a, l, r):
2     if r - l <= 1: return
```

```
    m = l + floor((r - l) / 2)
    p = DeterministicSelect(a, l, r, m)
    (lt, gt) = ThreeWayPartition(a, l, r, p)
    MedianQuickSort(a, l, lt)
    MedianQuickSort(a, gt, r)
```

Theorem 3.5. *Median quicksort is correct and takes $\Theta(n \log n)$ worst-case time in the comparison model, using $\mathcal{O}(\log n)$ recursion depth.*

Proof. Partition correctness and induction prove sorting correctness exactly as for ordinary quicksort. BFPRT plus partition costs $\Theta(n)$ at a node, and the median leaves two sides of size at most $n/2$. Thus

$$Q(n) \le Q(\lfloor n/2 \rfloor) + Q(\lceil n/2 \rceil - 1) + cn = \mathcal{O}(n \log n)$$

by the level-sum argument of chapter 2. The comparison-sorting lower bound supplies $\Omega(n \log n)$, so the time is $\Theta(n \log n)$. Halving at each recursive step gives depth $\mathcal{O}(\log n)$. □

This construction is theoretically important: it shows that the quicksort partition paradigm itself is not responsible for quadratic worst cases. In practice, introsort usually obtains the same worst-case asymptotic bound with smaller constants by switching from ordinary quicksort to heapsort when recursion becomes too deep.

3.4 Stable counting sort and sorting pairs

EXAMINATION SYLLABUS: ITEM 15

Comparison lower bounds do not apply when integer keys may be used as array indices. Assume every key lies in $0, \dots, K-1$.

Listing 3.4: Stable counting sort

```
function StableCountingSort(a[0..n-1], key, K):
    count[0..K-1] = all zeros
    for record in a:
        count[key(record)] = count[key(record)] + 1

    start[0] = 0
    for v = 1 to K - 1:
        start[v] = start[v - 1] + count[v - 1]

    next = start
    output[0..n-1]
    for record in a from left to right:
        v = key(record)
        output[next[v]] = record
        next[v] = next[v] + 1
    return output
```

Theorem 3.6 (Counting-sort correctness and stability)**.** *The output is a stable nondecreasing permutation of the input. The running time is $\Theta(n + K)$, and additional memory is $\Theta(n + K)$.*

Proof. After the first loop, `count[v]` is exactly the multiplicity of key v. Thus

$$\text{start}[v] = \sum_{u<v} \text{count}[u]$$

is the first output position reserved for key v. The positions

$$[\text{start}[v], \text{start}[v] + \text{count}[v])$$

are disjoint, consecutive, and appear in increasing v. Every input record is placed once into the next free position of its key's interval, proving multiset preservation and sortedness. Equal-key records are scanned and placed from left to right, so their relative order is preserved. The loops inspect n, K, n objects respectively, proving the time bound; the arrays have the stated sizes. □

If stability is unnecessary and records are merely integer keys, one may output each value v exactly `count[v]` times using only $\Theta(K)$ auxiliary memory. Stability requires either an output array or a more sophisticated in-place method.

3.4.1 Lexicographic pairs

Suppose records have a pair of integer keys (x, y) and must be ordered lexicographically: x is primary and y secondary. First stably sort by y, then stably sort the resulting sequence by x.

Proposition 3.7 (Stable two-pass sorting)**.** *The two stable passes produce lexicographic order by (x, y).*

Proof. After the second pass, records are grouped in nondecreasing x. Inside any group with equal x, stability preserves the relative order produced by the first pass, which is nondecreasing y. Therefore every pair is in lexicographic order. Both passes preserve all records. □

If $x \in [0, K_x)$ and $y \in [0, K_y)$, the time is $\Theta(n + K_x + K_y)$, not $\Theta(K_x K_y)$.

3.5 Least-significant-digit radix sort

EXAMINATION SYLLABUS: ITEM 16

Write every nonnegative key using exactly d base-B digits,

$$x = \sum_{j=0}^{d-1} x_j B^j, \qquad 0 \le x_j < B,$$

padding with leading zeros. LSD radix sort stably sorts by digit x_0, then x_1, and so on through x_{d-1}.

Listing 3.5: LSD radix sort

```
procedure LSDRadixSort(a, B, d):
    power = 1
    for j = 0 to d - 1:
        StableCountingSort(a,
            key = function(x) return floor(x / power) mod B,
            K = B)
        power = power * B
```

Theorem 3.8 (LSD invariant and correctness)**.** *After pass j, the array is stably ordered by the tuple $(x_j, x_{j-1}, \ldots, x_0)$, equivalently by the residue $x \bmod B^{j+1}$. After all d passes, it is sorted by the full key.*

Proof. Induct on j. The first stable counting-sort pass orders by x_0. Assume the claim after pass $j-1$. Pass j groups records by nondecreasing x_j. Within an equal-x_j group, stability preserves the previous order by $(x_{j-1}, \ldots, x_0)$. Hence the combined tuple is ordered. For $j = d-1$, this tuple contains all digits and determines the numeric order. □

Each pass takes $\Theta(n + B)$ time, so total time is $\Theta(d(n + B))$, with $\Theta(n + B)$ working memory reusable between passes. For w-bit keys, choosing $B = 2^b$ gives $d = \lceil w/b \rceil$. If $B = \Theta(n)$, the time is $\Theta(n \lceil w/\log n \rceil)$; for $w = \mathcal{O}(\log n)$ this is linear. Signed integers require a monotone encoding, such as flipping the sign bit in two's-complement representation before sorting.

Common pitfall 3.9. Processing digits from most significant to least significant with ordinary stable global passes is not the same algorithm: a later low-digit pass would destroy the priority of earlier high digits. MSD radix sort is instead recursive within digit buckets.

Part II

Linear Data Structures and Heaps

LECTURE UNIT 4

Lists, Stacks, and Queues

4.1 Singly and doubly linked lists

EXAMINATION SYLLABUS: ITEM 17

An array stores consecutive elements and supports random access; a linked list stores nodes at arbitrary addresses and records adjacency explicitly.

Definition 4.1 (Singly linked list)**.** A node contains a value and a pointer `next`. A list stores a pointer `head`; following `next` pointers visits every node exactly once and eventually reaches `null`. This acyclic reachability property is the list invariant.

Insertion after a known node u is

Listing 4.1: Core singly linked-list mutations

```
struct Node { int value; Node* next; };

void pushFront(Node*& head, int x) {
    head = new Node{x, head};
}

void insertAfter(Node* u, int x) {
    u->next = new Node{x, u->next};
}

void eraseAfter(Node* u) {              // requires u->next != nullptr
    Node* victim = u->next;
    u->next = victim->next;
    delete victim;
}
```

Each mutation changes only a constant number of pointers, hence takes $\Theta(1)$ time when the location is known. Finding the i-th node or the predecessor of a given node generally takes $\Theta(n)$, because pointers expose only the next node.

Proposition 4.2 (Singly linked-list mutation correctness)**.** *Assuming the list invariant before any listed operation and its preconditions, the operation preserves the invariant and has the intended sequence effect.*

Proof. For insertion, the new node points to the old successor before the predecessor is redirected to the new node. Thus the unique path is changed from $u \to v$ to $u \to \text{new} \to v$; no existing suffix is lost and no cycle is created. For deletion, redirecting u to the victim's successor removes exactly the victim from the reachability path. The victim is freed only after its successor was saved through the new link. Front insertion is the same argument with an implicit sentinel preceding the head. □

Definition 4.3 (Doubly linked list)**.** Each node stores `prev` and `next`. For every adjacent pair u, v,

$$u.\text{next} = v \quad \Longleftrightarrow \quad v.\text{prev} = u.$$

Sentinel nodes `root` often represent both ends: an empty circular list has `root.next = root.prev = &root`.

With sentinels, insertion before a known node v and deletion of a known node v are uniform.

Listing 4.2: Doubly linked-list mutations

```
struct DNode { int value; DNode* prev; DNode* next; };

void insertBefore(DNode* v, DNode* x) {
    x->prev = v->prev;  x->next = v;
    v->prev->next = x;  v->prev = x;
}

void erase(DNode* v) {                  // v is not the sentinel
    v->prev->next = v->next;
    v->next->prev = v->prev;
    delete v;
}
```

The four link equalities can be checked directly; hence known-node insertion and deletion are $\Theta(1)$. A doubly linked list spends one extra pointer per node to permit constant-time predecessor access and deletion by pointer.

4.2 A stack implemented by a list

EXAMINATION SYLLABUS: ITEM 18

Definition 4.4 (Stack ADT)**.** A stack is a last-in, first-out sequence supporting `push(x)`, `top()`, and `pop()`. The latter two require a nonempty stack.

Store the stack top at the head of a singly linked list. `push` is front insertion; `pop` removes the head; `top` reads its value. All take $\Theta(1)$ worst-case time and the representation uses $\Theta(s)$ words for s elements.

Theorem 4.5 (Stack representation invariant)**.** *If list order from head to tail equals stack order from top to bottom before an operation, it does so afterward, and every stack operation has the specified effect.*

Proof. Front insertion places the new element before all older elements, exactly as `push` requires. Head removal returns and removes the unique most recent element. Reading the head changes nothing. The list-mutation proof ensures structural validity. □

4.3 Balanced brackets with several bracket types

EXAMINATION SYLLABUS: ITEM 19

Problem. Given a string over opening and closing symbols such as (exttt( ) [ ] { }), decide whether every close matches the most recent unmatched open of the same type and whether no opens remain unmatched.

Listing 4.3: Bracket validation

```
function IsBalanced(s):
    stack = empty
    for character c in s:
        if c is an opening bracket:
            stack.push(c)
        else if c is a closing bracket:
            if stack is empty or type(stack.top) != type(c):
                return false
            stack.pop()
    return stack is empty
```

Theorem 4.6. *The algorithm returns true exactly for correctly nested bracket strings.*

Proof. After processing any prefix, maintain the invariant that the stack, bottom to top, contains exactly the unmatched opening brackets of that prefix in their occurrence order. An opening bracket appends one unmatched open. A closing bracket can be valid only if it matches the most recent unmatched open: nesting forces this open to close before any earlier one. The algorithm checks precisely this condition and then removes the matched open. Therefore an early rejection certifies an invalid prefix. If the scan finishes, validity is equivalent to having no unmatched opens, which is exactly stack emptiness. □

Each character is pushed and popped at most once, so time is $\Theta(n)$ and stack memory is $\mathcal{O}(n)$, tight for a prefix of n opening brackets.

4.4 Nearest smaller or greater elements

EXAMINATION SYLLABUS: ITEM 20

Consider the nearest strictly smaller element to the left:

$$L[i] = \max\{j < i : a[j] < a[i]\},$$

with $L[i] = -1$ if the set is empty. A monotone stack solves all n searches in linear time.

Listing 4.4: Nearest strictly smaller element to the left

```
stack = empty stack of indices
```

```
for i = 0 to n - 1:
    while stack is not empty and a[stack.top] >= a[i]:
        stack.pop()
    L[i] = -1 if stack is empty else stack.top
    stack.push(i)
```

Invariant 4.7. Immediately before processing i, stack indices increase from bottom to top, their values increase strictly, and every index removed earlier is permanently dominated for every future query by a later index of no greater value.

Theorem 4.8 (Monotone-stack correctness)**.** *The algorithm computes the nearest strictly smaller index to the left for every position in* $\Theta(n)$ *time.*

Proof. Popping all values at least $a[i]$ leaves, if anything, a top j with $a[j] < a[i]$. Suppose a later index k, $j < k < i$, were also smaller. It could not have been popped by i, since popping requires a later value no larger and the final top is the latest surviving index; tracing the pop that removed k would give an even later surviving dominator. Thus the top is the nearest valid index. If the stack is empty, every prior candidate was removed by a later value no larger than it and ultimately no prior value below $a[i]$ survives; none can be valid.

Popping values at least the current one preserves strictly increasing stack values; pushing i preserves index order. Each index is pushed once and popped at most once, so all while-loop iterations total at most n, giving $\Theta(n)$ time and $\mathcal{O}(n)$ memory. □

The four variants follow mechanically:

Wanted relation	Scan direction	Pop while
smaller on left	left to right	$a[top] \geq a[i]$
greater on left	left to right	$a[top] \leq a[i]$
smaller on right	right to left	$a[top] \geq a[i]$
greater on right	right to left	$a[top] \leq a[i]$

Use a strict or non-strict pop condition according to whether equal values qualify.

4.5 Evaluating reverse Polish notation

EXAMINATION SYLLABUS: ITEM 21

In reverse Polish notation (RPN), operands precede their operator. For example, $extttt234*+$ denotes $2 + 3 \cdot 4$. Suppose operators have fixed arity; for simplicity, the pseudocode shows binary operators.

Listing 4.5: RPN evaluation

```
function EvaluateRPN(tokens):
    values = empty stack
    for token t in tokens:
```

```
        if t is an operand:
            values.push(value(t))
        else:
            if values.size < 2: error MALFORMED
            right = values.pop()
            left  = values.pop()
            values.push(apply(t, left, right))
    if values.size != 1: error MALFORMED
    return values.top
```

Theorem 4.9 (RPN evaluation correctness)**.** *For a well-formed RPN expression, the algorithm returns its value. It rejects every token sequence that cannot form one complete binary expression.*

Proof. After every prefix, the stack contains, from bottom to top, the values of the maximal complete subexpressions in that prefix that have not yet been consumed by a later operator. An operand forms a new one-token subexpression. A binary operator must consume the two most recent subexpressions, with the earlier as its left operand and the later as its right; popping in the stated order and pushing the result preserves the invariant. Fewer than two values makes the prefix impossible. At the end, exactly one maximal expression is necessary and sufficient for the whole sequence to represent one expression. □

For n tokens, time is $\Theta(n)$ and memory is $\mathcal{O}(n)$. Noncommutative operators make the pop order essential.

4.6 A stack with constant-time minimum

Examination syllabus: item 22

Augment every stored element x with the minimum of the stack at the moment it is pushed:

$$(x, m), \qquad m = \min\{x, \text{old minimum}\}.$$

For the first element, set $m = x$.

Listing 4.6: Minimum stack

```
class MinStack {
    vector<pair<int,int>> s;           // (value, minimum through this entry)
public:
    void push(int x) {
        int m = s.empty() ? x : min(x, s.back().second);
        s.push_back({x, m});
    }
    void pop() { s.pop_back(); }
    int top() const { return s.back().first; }
    int getMin() const { return s.back().second; }
};
```

Proposition 4.10. *The second component at every depth equals the minimum of all values at or below that depth. Thus all four operations take* $\Theta(1)$ *worst-case time and the structure uses* $\Theta(s)$ *words.*

Proof. Induct on the number of pushes. The first stored minimum is its only value. A new stack minimum is exactly the smaller of the new value and the old stack minimum. Popping exposes a pair whose stored statement was already true and whose prefix is unchanged. Therefore the top pair always stores the current minimum. □

An alternative stores only values that become new minima, together with duplicate counts; it can save memory but has the same asymptotic bounds.

4.7 A queue implemented by a list

EXAMINATION SYLLABUS: ITEM 23

Definition 4.11 (Queue ADT)**.** A queue is a first-in, first-out sequence supporting `pushBack(x)`, `front()`, and `popFront()`.

Maintain pointers to both head and tail of a singly linked list. Enqueue links a new node after the tail and moves `tail`; dequeue removes `head`. When the last node is removed, both pointers become null.

Theorem 4.12. *List order from head to tail equals queue order from oldest to newest. With both end pointers, every queue operation takes* $\Theta(1)$ *worst-case time.*

Proof. Appending after the unique tail places a new item after all existing items. Removing the head removes the unique oldest item. These operations preserve the order invariant. Only a constant number of pointers changes; the empty/nonempty boundary case resets both ends. Without a tail pointer, enqueue would require a linear traversal. □

4.8 A queue on two stacks and a minimum queue

EXAMINATION SYLLABUS: ITEM 24

Maintain two stacks: `in` receives new values, while `out` supplies old values. To read or remove the front, if `out` is empty, move every element from `in` to `out`; the two reversals make the oldest element the top.

Listing 4.7: Queue on two stacks

```
procedure pushBack(x):
    in.push(x)

procedure refill():
    if out is empty:
```

```
        while in is not empty:
            out.push(in.pop())

function front(): refill(); return out.top()
procedure popFront(): refill(); out.pop()
```

Invariant 4.13. Queue order is the sequence obtained by reading `out` from top to bottom, followed by `in` from bottom to top.

Theorem 4.14 (Correctness and amortized cost)**.** *The two-stack structure implements a queue. Any sequence of m operations takes $\mathcal{O}(m)$ total time, although one operation may take $\Theta(n)$.*

Proof. Pushing appends an element to the second part of the invariant sequence. A refill reverses `in`, making its oldest element the top of `out`, while the abstract concatenated sequence is unchanged. Therefore `front` and `popFront` access the abstract first element.

For cost, charge each enqueued element for its push into `in`, at most one pop from `in`, at most one push into `out`, and at most one final pop. It is moved between stacks only when `out` is empty and never returns to `in`. Thus each element causes $\mathcal{O}(1)$ primitive stack operations and the total is $\mathcal{O}(m)$. □

To support `getMin`, make both `in` and `out` minimum stacks. The queue minimum is

$$\min\{\min(\texttt{in}), \min(\texttt{out})\},$$

ignoring an empty side. This is correct because the two stacks partition the queue elements. Enqueue, dequeue, and front remain $\mathcal{O}(1)$ amortized, while minimum is $\Theta(1)$ worst case.

LECTURE UNIT 5

Heaps, Amortization, Dynamic Arrays, and Sparse Tables

5.1 Binary heaps and their array representation

EXAMINATION SYLLABUS: ITEM 25

Definition 5.1 (Complete binary tree)**.** A complete binary tree has every level full except possibly the last, and the last level is filled from left to right. Its height is $\lfloor \log_2 n \rfloor$ when it has $n \geq 1$ nodes.

Number nodes in level order from 0. Then

$$\text{parent}(i) = \lfloor (i-1)/2 \rfloor, \quad \text{left}(i) = 2i+1, \quad \text{right}(i) = 2i+2,$$

whenever the indicated index is valid. Thus a complete tree needs no pointers; an array already encodes its shape.

Definition 5.2 (Min-heap)**.** A binary min-heap is a complete binary tree satisfying the **heap order**

$$\text{key}(\text{parent}(v)) \leq \text{key}(v)$$

for every non-root node v. Equal keys are allowed.

Proposition 5.3. *The root of a nonempty min-heap contains a global minimum.*

Proof. Every node v has a root-to-v path. Repeated heap inequalities along this path show $\text{key}(\mathit{root}) \leq \text{key}(v)$. □

Heap order is weaker than sorted order: nodes in different branches need not be comparable. This weakness permits logarithmic updates.

5.2 Sifting operations

EXAMINATION SYLLABUS: ITEM 26

5.2.1 Sift up

Suppose heap order may be violated only between a node i and its parent, while the two subtrees rooted at i are heaps. Repeatedly exchange i with its parent while its key is smaller.

Listing 5.1: Sift up in a min-heap

```
void siftUp(vector<int>& h, int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (h[p] <= h[i]) break;
        swap(h[p], h[i]);
        i = p;
    }
}
```

Lemma 5.4 (Sift-up correctness)**.** *Under the stated precondition, `siftUp` restores heap order and changes no keys. It takes $\mathcal{O}(\log n)$ time.*

Proof. Only the moving key x can violate order with its parent. When x is swapped up past a larger parent key y, placing y in the old position of x is safe: $y > x$, and the children there were at least x, but this alone does not imply they are at least y. The stronger precondition arising from insertion or decrease-key is that x was the only changed key; before the change, the old key at that position was at least y, and its children were at least the old key. Hence they are at least y. The sibling subtree is unchanged, and the only possible remaining violation is now between x and its new parent.

When the loop stops, x is at the root or no smaller than its parent, so no violation remains. Each iteration decreases depth by one, and heap height is $\lfloor \log n \rfloor$. □

For a freshly appended leaf, there are no children, so the same proof applies without the stronger decrease-key history.

5.2.2 Sift down

Suppose only a root i may be larger than one of its children, while both child subtrees are heaps. Exchange it with its smaller child until it is no larger than both children.

Listing 5.2: Sift down in a min-heap

```
void siftDown(vector<int>& h, int i, int size) {
    while (true) {
        int best = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        if (left < size && h[left] < h[best]) best = left;
        if (right < size && h[right] < h[best]) best = right;
        if (best == i) break;
        swap(h[i], h[best]);
        i = best;
    }
}
```

Lemma 5.5 (Sift-down correctness)**.** *Under the stated precondition,* `siftDown` *restores heap order in* $\mathcal{O}(\log n)$ *time.*

Proof. Let x be the moving key. If it exceeds a child, swapping with the smaller child y places y above both child roots: $y < x$ and y is no larger than its sibling by choice. Therefore the new parent position is valid. The sibling subtree is unchanged; within the chosen subtree the only possible violation is between x and its new children. This is the loop invariant. At a leaf or when x is no larger than both children, all edges satisfy heap order. Each swap increases depth by one, so there are at most $\lfloor \log n \rfloor$ swaps. □

5.3 Priority-queue operations

Examination syllabus: item 27

The binary heap implements a min-priority queue.

Operation	Implementation	Time
`getMin`	return array element 0	$\Theta(1)$
`insert(x)`	append x, then sift up	$\mathcal{O}(\log n)$
`extractMin`	save root, move last key to root, remove last, sift down	$\mathcal{O}(\log n)$
`decreaseKey(i,x)`	require $x \le h[i]$; assign, then sift up	$\mathcal{O}(\log n)$

Theorem 5.6 (Operation correctness)**.** *Each operation returns the abstract priority-queue result and preserves the heap invariant.*

Proof. The root-minimum proposition proves `getMin`. Appending a leaf preserves completeness and can violate only its parent edge; sift up repairs it. Removing the last array cell preserves completeness. Moving that key to the root can violate only downward edges, repaired by sift down; the saved root was a minimum. Decreasing one key cannot violate its child edges but may violate its parent edge, so sift up is sufficient. The sifting lemmas supply preservation and time. □

A handle to an element must track its current array index. Every swap must update both affected handles; otherwise `decreaseKey` by handle is incorrect.

5.4 Linear-time heap construction

Examination syllabus: item 28

Inserting n elements separately gives $\mathcal{O}(n \log n)$. Floyd's `heapify` does better: leaves are already heaps, so sift down every internal node in reverse level order.

Listing 5.3: Floyd heap construction

```
1 void heapify(vector<int>& h) {
2     for (int i = static_cast<int>(h.size()) / 2 - 1; i >= 0; --i)
```

```
        siftDown(h, i, static_cast<int>(h.size()));
}
```

Theorem 5.7 (Heapify correctness)**.** *When iteration i begins, both child subtrees of i are heaps. After the final iteration, the entire array is a heap.*

Proof. Nodes with index at least $\lfloor n/2 \rfloor$ are leaves. Processed indices decrease. Both children of i, when present, have larger indices and have already been processed; their subtrees are heaps. The sift-down lemma therefore makes the subtree rooted at i a heap. Reverse induction reaches root 0, whose subtree is the entire tree. □

Theorem 5.8 (Linear heapify time)**.** *Floyd heap construction takes $\Theta(n)$ time.*

Proof. A node of height h can move down at most h levels. In a complete binary tree, at most $\left\lceil n/2^{h+1} \right\rceil$ nodes have height exactly h. Thus total work is at most

$$c \sum_{h \geq 0} \left\lceil \frac{n}{2^{h+1}} \right\rceil h = \mathcal{O}(n) + cn \sum_{h \geq 0} \frac{h}{2^{h+1}}.$$

To evaluate the series, for $|x| < 1$, differentiating $\sum_{h \geq 0} x^h = 1/(1-x)$ gives $\sum_{h \geq 1} h x^{h-1} = 1/(1-x)^2$. At $x = 1/2$, $\sum_{h \geq 0} h/2^{h+1} = 1$. Hence the upper bound is $\mathcal{O}(n)$. Reading the array and considering its internal nodes costs $\Omega(n)$, so the time is $\Theta(n)$. □

The reason is structural: half the nodes are leaves and cost zero, one quarter can move only one level, one eighth only two, and so forth.

5.5 In-place heapsort and a priority-queue impossibility

EXAMINATION SYLLABUS: ITEM 29

For ascending order, build a *max*-heap. Repeatedly exchange the maximum at index 0 with the last position of the current heap, shrink the heap by one, and sift the new root down. Max-heap code reverses comparisons in min-heap code.

Listing 5.4: In-place heapsort

```
procedure HeapSort(a):
    build a max-heap by Floyd heapify
    heapSize = n
    while heapSize > 1:
        swap(a[0], a[heapSize - 1])
        heapSize = heapSize - 1
        siftDownMax(a, 0, heapSize)
```

Invariant 5.9. Before each loop iteration, $a[0..heapSize)$ is a max-heap, $a[heapSize..n)$ is nondecreasing, and every key in the heap prefix is at most every key in the sorted suffix.

Theorem 5.10 (Heapsort correctness and bounds)**.** *Heapsort returns a nondecreasing permutation in* $\Theta(n \log n)$ *worst-case time and uses* $\Theta(1)$ *additional words in its iterative form.*

Proof. Heapify establishes the invariant with an empty suffix. The max-heap root is the largest prefix key. Swapping it into the suffix's first position makes the new suffix sorted and no smaller than the remaining prefix. Shrinking preserves the multiset; sift down restores the prefix heap. At size one, the whole array is sorted.

Heapify is $\Theta(n)$. There are $n - 1$ extractions, each $\mathcal{O}(\log n)$, for $\mathcal{O}(n \log n)$; the comparison-sorting lower bound gives $\Omega(n \log n)$. All operations occur inside the array with constant scalar state. □

Heapsort is generally not stable: distant swaps may reverse equal records.

Theorem 5.11 (No constant-time comparison heap for both updates)**.** *No comparison-based priority queue can support both* `insert` *and* `extractMin` *in* $\mathcal{O}(1)$ *amortized time for every operation sequence.*

Proof. If it could, insert n arbitrary distinct keys and then call `extractMin` n times. The returned sequence sorts the keys. The assumed total time is $\mathcal{O}(n)$, so the number of comparisons is $\mathcal{O}(n)$, contradicting the $\Omega(n \log n)$ comparison-sorting lower bound. The argument also rules out both operations being $\mathcal{O}(1)$ worst case. □

One operation can be constant: an unsorted array gives $\Theta(1)$ insertion and $\Theta(n)$ extraction; a sorted representation reverses the tradeoff.

5.6 Deleting from a binary heap

5.6.1 Deletion by pointer or handle

EXAMINATION SYLLABUS: ITEM 30

Assume a handle tells the current array index i. Swap the target with the last element, remove the last cell, then repair in the only possible direction.

Listing 5.5: Delete a heap element at known index

```
procedure EraseAt(h, i):
    swapWithHandleUpdates(h[i], h[n - 1])
    remove h[n - 1]
    if i == new n: return
    if i > 0 and h[i] < h[parent(i)]:
        siftUp(h, i)
    else:
        siftDown(h, i)
```

Theorem 5.12. *Deletion by a valid handle preserves heap order in* $\mathcal{O}(\log n)$ *time.*

Proof. Removing the last cell preserves completeness. Only the moved key can violate an edge. If it is smaller than its parent, its child edges are safe: before the move, its children were descendants whose keys are at least the deleted key's old local constraints, and the standard sift-up precondition is satisfied along the repair path. Sift up repairs the upward violation. Otherwise the parent edge is safe, and only child edges may fail, so sift down repairs them. Each path has logarithmic length. All swaps update handles, preserving their index invariant. □

An equally simple method decreases the key to a sentinel $-\infty$, sifts it to the root, and calls `extractMin`; this requires a sentinel below every legal key and performs two logarithmic walks.

5.6.2 Deletion by value

EXAMINATION SYLLABUS: ITEM 31

With only a binary heap array and a value x, first locate an equal cell, then call `EraseAt`. Location is $\mathcal{O}(n)$, so total worst-case time is $\mathcal{O}(n)$.

Proposition 5.13 (Linear search is necessary in a bare heap)**.** *In the comparison model, finding whether an arbitrary value x occurs in a min-heap may require $\Omega(n)$ comparisons.*

Proof. Consider a heap whose root is below x and whose remaining nodes, especially the $\lceil n/2 \rceil$ leaves, are all above the root but otherwise unconstrained. Heap order provides no ordering among the leaves. An adversary can answer that each inspected leaf is not x and place x, if present, at the last uninspected leaf without violating heap order. Therefore linearly many candidates may need inspection. □

To accelerate deletion, maintain a dictionary from each value to a set of its current heap indices. Every heap swap removes and adds two indices in those sets. With balanced sets, update and deletion take $\mathcal{O}(\log n)$ worst-case; with hash sets they take expected $\mathcal{O}(1)$ per index update and $\mathcal{O}(\log n)$ for sifting overall. Duplicates require a set or multiset of indices, not a single index.

5.7 Binomial trees and binomial heaps

EXAMINATION SYLLABUS: ITEM 32

Definition 5.14 (Binomial tree)**.** The binomial tree B_0 is one node. For $k \geq 1$, B_k is formed by linking the root of one B_{k-1} as the leftmost child of the root of another B_{k-1}.

Lemma 5.15 (Structure of B_k)**.** *B_k has 2^k nodes, height k, root degree k, and exactly $\binom{k}{d}$ nodes at depth d.*

Proof. Induct on k. Linking two copies doubles node count, adds one to maximum depth, and adds one root child. For depth counts, one copy contributes $\binom{k-1}{d}$; the attached copy is shifted down and contributes $\binom{k-1}{d-1}$. Pascal's identity gives $\binom{k}{d}$. □

Definition 5.16 (Binomial min-heap)**.** A binomial heap is a forest of min-heap-ordered binomial trees with at most one tree of each degree. Roots are stored in increasing degree order.

If the heap has $n = \sum_k b_k 2^k$ nodes, its trees are exactly the B_k for which binary digit $b_k = 1$. Hence it has at most $\lfloor \log n \rfloor + 1$ roots. Its advantage over a binary heap is merge: two forests combine like binary numbers in $\mathcal{O}(\log(n+m))$, whereas melding two unrelated binary-heap arrays requires $\Theta(n+m)$ rebuilding or movement in the standard representation.

5.8 Binomial-heap operations

EXAMINATION SYLLABUS: ITEM 33

Link. Two heap-ordered B_k trees are linked by making the larger-key root a child of the smaller-key root. Heap order is preserved and the result is B_{k+1}.

Merge. Merge root lists by degree. Whenever two trees have the same degree, link them; handle a possible third tree like a binary carry. There are $\mathcal{O}(\log(n+m))$ degrees.

Theorem 5.17 (Merge correctness)**.** *Binomial-heap merge produces a heap containing exactly the union of keys, with at most one tree of each degree, in $\mathcal{O}(\log(n+m))$ time.*

Proof. Process degrees from low to high. The invariant is that all lower degrees have been normalized to at most one tree and any extra tree is a single carry of the current degree. Zero or one current tree is emitted; two are linked into one carry of next degree; three emit one and link two. Linking preserves heap order and multiset, and exactly mirrors binary addition of the node counts. Only logarithmically many degree positions exist. □

Other operations reduce to merge or a root-path repair.

Operation	Method	Worst-case time
`insert(x)`	merge with singleton B_0	$\mathcal{O}(\log n)$
`getMin`	scan roots; or maintain a minimum-root pointer	$\mathcal{O}(\log n)$, or $\Theta(1)$
`extractMin`	find/remove minimum root, reverse its children into a heap, merge	$\mathcal{O}(\log n)$
`decreaseKey(v,x)`	lower key and exchange it upward while below parent	$\mathcal{O}(\log n)$

For extraction, the removed root's children have degrees $k-1, k-2, \ldots, 0$; reversing their list restores increasing degree order. Each child subtree was already heap ordered. Merge then restores the forest invariant. For decrease-key, only the parent edge can be violated and height is logarithmic. If external handles denote logical records, exchange whole records or update handles whenever keys move.

5.9 Amortized analysis and accounting time

EXAMINATION SYLLABUS: ITEM 34

Definition 5.18 (Amortized bound)**.** Let actual costs of an arbitrary legal operation sequence be $c_1, \dots, c_m$. Numbers $\widehat{c}_i$ are valid amortized costs if, for every prefix,

$$\sum_{i=1}^{t} c_i \le \sum_{i=1}^{t} \widehat{c}_i \qquad (1 \le t \le m).$$

If every $\widehat{c}_i = \mathcal{O}(f)$, then total actual cost is $\mathcal{O}(mf)$.

Amortized time is a worst-case guarantee over operation sequences. It does not mean average input, expected time, or "usually fast." Three equivalent proof styles are common.

1. **Aggregate method**: directly bound total cost of m operations.
2. **Accounting method**: overcharge cheap operations and store credits that pay for later expensive work.
3. **Potential method**: choose $\Phi(D) \ge 0$ for state D and define $\widehat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.

For the potential method, telescoping gives

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \widehat{c}_i - \Phi(D_m) + \Phi(D_0) \le \sum_{i=1}^{m} \widehat{c}_i + \Phi(D_0).$$

When $\Phi(D_0) = 0$, nonnegative final potential proves the amortized bound.

5.10 The accounting method: two-stack queue

EXAMINATION SYLLABUS: ITEM 35

Return to the queue of chapter 4. Charge three coins to every `pushBack`:

1. one pays for pushing the element into `in`;
2. two remain attached to the element and later pay for popping it from `in` and pushing it into `out`.

Charge one coin to `popFront` to pay for popping from `out`; charge a constant to `front`. Credits are never negative because an element is moved only after it was inserted and received its two stored coins. Each element moves at most once. Thus every abstract operation has constant accounting cost, and every sequence of m operations costs $\mathcal{O}(m)$, even though a refill may use linear actual time.

This proof is stronger than dividing one expensive operation by a guessed number of future operations: credits identify precisely which earlier operations pay and establish that the budget exists before it is spent.

5.11 Dynamic arrays (vectors)

EXAMINATION SYLLABUS: ITEM 36

A dynamic array stores s elements in a contiguous allocation of capacity $C \geq s$. It supports $\Theta(1)$ random access. To append when $s = C$, allocate capacity $2C$, copy all C elements, free the old allocation, and append. Begin with capacity 1.

Invariant 5.19. The first s cells contain the logical sequence in order; cells $s, \ldots, C-1$ are spare; and $0 \leq s \leq C$.

Theorem 5.20 (Amortized append)**.** *A sequence of m appends to an initially empty doubling vector takes $\Theta(m)$ total time. Hence append costs $\Theta(1)$ amortized.*

Proof. Ordinary writes cost m. Reallocations copy arrays of sizes $1, 2, 4, \ldots, 2^q < m$. Their total is the geometric sum

$$1 + 2 + \cdots + 2^q = 2^{q+1} - 1 < 2m.$$

Thus total work is below a constant times m. The m required element writes give the matching lower bound. □

Insertion or deletion at position i shifts $\Theta(s-i)$ elements, so it is $\mathcal{O}(n)$ and can be $\Theta(n)$, independently of reallocation. Removing the last element is $\Theta(1)$. If memory should shrink, halve capacity only when $s \leq C/4$; after shrinking, $s \leq C/2$, so linearly many updates are needed before the next resize. Shrinking immediately at $s = C/2$ can thrash under alternating append/pop operations.

The contiguity that enables random access and cache locality also invalidates raw pointers and iterators whenever reallocation occurs.

5.12 Sparse tables for static range minimum

EXAMINATION SYLLABUS: ITEM 37

Model problem. Preprocess a static array so that

$$\mathrm{RMQ}(l, r) = \min_{l \leq i < r} a[i]$$

is answered in $\Theta(1)$ time.

Define

$$st[k][i] = \min a[i..i + 2^k),$$

for every block fitting inside the array. The recurrence is

$$st[0][i] = a[i], \qquad st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}]).$$

Listing 5.6: Sparse-table query

```
function RangeMinimum(l, r):            // l < r
    k = floor(log2(r - l))
    return min(st[k][l], st[k][r - 2^k])
```

Precompute $\lfloor \log_2 x \rfloor$ for $x = 1, \ldots, n$ in linear time, or obtain it from a word instruction.

Theorem 5.21 (Sparse-table correctness and complexity)**.** *Construction takes $\Theta(n \log n)$ time and memory, and every nonempty RMQ is answered correctly in $\Theta(1)$ time.*

Proof. Induction on k proves each table entry: a length-2^k interval is the union of its two consecutive length-2^{k-1} halves. There are $\lfloor \log n \rfloor + 1$ levels with $\mathcal{O}(n)$ entries each, proving the build bounds.

For a query of length L, let $k = \lfloor \log L \rfloor$. The two intervals

$$[l, l + 2^k), \qquad [r - 2^k, r)$$

both lie inside $[l, r)$, and since $2^{k+1} > L$, their union covers the query. They may overlap, but minimum is idempotent: counting an element twice does not change the result. Therefore the minimum of the two stored minima is precisely the query minimum. □

The same overlapping-block argument works for any associative idempotent operation, such as maximum, gcd, or bitwise AND. It fails for sum because overlap would be counted twice; a disjoint power-of-two decomposition then needs $\mathcal{O}(\log n)$ blocks, or prefix sums if inverses are available.

Part III

Range-Query Data Structures

LECTURE UNIT 6

Range-Query Data Structures

6.1 Segment trees: point updates and range queries

EXAMINATION SYLLABUS: ITEM 38

Model problem. Maintain an array under point assignments and answer range sums

$$\mathrm{sum}(l, r) = \sum_{i=l}^{r-1} a[i].$$

Unlike prefix sums, a segment tree keeps updates and queries logarithmic.

Pad the array to $N = 2^{\lceil \log_2 n \rceil}$ with zeros. The root represents $[0, N)$; a node for $[L, R)$ has children $[L, M)$ and $[M, R)$, where $M = (L + R)/2$. Store the sum of its interval. In an iterative array layout, leaves occupy indices $N, \ldots, 2N - 1$, and a parent v has children $2v$ and $2v + 1$.

Invariant 6.1 (Segment-tree aggregate)**.** For every node v representing I_v,

$$tree[v] = \sum_{i \in I_v} a[i].$$

Listing 6.1: Iterative sum segment tree

```
class SegmentTree {
    int N;
    vector<long long> t;
public:
    explicit SegmentTree(const vector<int>& a) {
        N = 1; while (N < (int)a.size()) N *= 2;
        t.assign(2 * N, 0);
        for (int i = 0; i < (int)a.size(); ++i) t[N + i] = a[i];
        for (int v = N - 1; v >= 1; --v) t[v] = t[2*v] + t[2*v+1];
    }
    void assignPoint(int i, long long x) {
        int v = N + i; t[v] = x;
        for (v /= 2; v >= 1; v /= 2) {
            t[v] = t[2*v] + t[2*v+1];
            if (v == 1) break;
        }
    }
    long long rangeSum(int l, int r) const { // [l, r)
        long long left = 0, right = 0;
        for (l += N, r += N; l < r; l /= 2, r /= 2) {
```

```
            if (l & 1) left += t[l++];
            if (r & 1) right = t[--r] + right;
        }
        return left + right;
    }
};
```

Theorem 6.2 (Segment-tree correctness)**.** *Construction establishes the aggregate invariant. Point assignment preserves it, and* `rangeSum(l,r)` *returns the sum on* $[l, r)$*.*

Proof. Leaves contain the corresponding values or padding identity 0. Reverse-order construction sets every internal node to the sum of two disjoint child intervals, so structural induction proves the invariant. A point assignment changes exactly one leaf; recomputing its ancestors bottom-up restores every and only affected aggregate.

For a query, the current index interval represents the not-yet-accounted portion. When l is a right child, its whole node interval belongs to the answer while its sibling lies outside, so it is consumed and l moves right. When r is a right boundary, decrementing it selects the left sibling immediately inside the range. After these boundary nodes are removed, dividing both indices by two coarsens the remaining aligned range to parents. Selected node intervals are disjoint and their union is exactly the original query; the two accumulators keep their left-to-right order, relevant for a noncommutative associative operation. At $l = r$, nothing remains. □

The tree has fewer than $2N < 4n$ nodes, so construction time and memory are $\Theta(n)$. An update visits one root-to-leaf path, $\Theta(\log n)$. A query selects at most two nodes at each tree level, hence $\mathcal{O}(\log n)$ time. Sums can be replaced by any associative operation with an identity, such as minimum, gcd, matrix multiplication, or function composition; order must be respected when the operation is noncommutative.

6.2 Binary descent in a segment tree

EXAMINATION SYLLABUS: ITEM 39

Aggregates can guide a search, not merely answer a completed query. Assume $a[i] \geq 0$. To find the least index r satisfying

$$\sum_{i=0}^{r} a[i] \geq x,$$

first reject if the root sum is below x. At node v, compare x with the left-child sum. If the left sum is at least x, descend left; otherwise subtract it from x and descend right.

Listing 6.2: First prefix whose sum reaches a threshold

```
function FirstPrefixAtLeast(x):
    if x <= 0: return 0
    if tree[root] < x: return NOT_FOUND
    v = root
```

```
    while v is not a leaf:
        if tree[left(v)] >= x:
            v = left(v)
        else:
            x = x - tree[left(v)]
            v = right(v)
    return index represented by v
```

Theorem 6.3 (Binary-descent correctness)**.** *For nonnegative array values, the algorithm returns the smallest index whose inclusive prefix sum is at least the original $x > 0$, or correctly reports that none exists, in $\Theta(\log n)$ time.*

Proof. At a current node interval $I = [L, R)$, maintain that all positions before L have already contributed a known amount, the current x is the remaining sum needed, and the smallest valid answer lies in I. If the left child contains sum at least x, nonnegativity implies a prefix inside it reaches x, so the smallest answer lies left. Otherwise no prefix ending in the left child reaches the target; every answer in the right child includes the entire left sum, which is subtracted. The invariant continues. At a leaf, its position is the unique candidate. Root sum below x proves nonexistence. One node is visited per level. □

For a fixed left endpoint l, seek the smallest $r \geq l$ with $\sum_{i=l}^{r} a[i] \geq x$. Traverse the canonical segment-tree nodes covering $[l, n)$ from left to right. Skip a whole node and subtract its sum while that sum is below the remaining x; at the first node large enough, perform the same binary descent inside it. The canonical cover contains $\mathcal{O}(\log n)$ boundary nodes and descent takes $\mathcal{O}(\log n)$, so total time remains $\mathcal{O}(\log n)$.

Common pitfall 6.4. Nonnegativity is essential: with negative values, a segment's total being below x does not imply that none of its prefixes reaches x. One must instead store a richer aggregate, for example total sum together with maximum prefix sum. For locating an arbitrary subarray of sum at least x, one may store total, maximum prefix, maximum suffix, and maximum subarray sum, then descend according to those values.

6.3 Lazy propagation: range assignment

Examination syllabus: item 40

Problem. Support assignment $a[i] \leftarrow x$ for every $i \in [l, r)$ and range-sum queries. Updating every leaf would be linear. Lazy propagation records the effect at a fully covered internal node and postpones work on its descendants.

Each node v for interval I_v stores:

- `sum[v]`, the correct logical sum on I_v;
- a Boolean `hasSet[v]`;
- if set, `setValue[v]`, meaning every logical leaf in I_v equals this value even if child fields are stale.

Listing 6.3: Lazy assignment primitives

```
procedure ApplySet(v, L, R, x):
    sum[v] = x * (R - L)
    hasSet[v] = true
    setValue[v] = x

procedure Push(v, L, R):
    if not hasSet[v] or R - L == 1: return
    M = (L + R) / 2
    ApplySet(left(v),  L, M, setValue[v])
    ApplySet(right(v), M, R, setValue[v])
    hasSet[v] = false

procedure Assign(v, L, R, ql, qr, x):
    if qr <= L or R <= ql: return
    if ql <= L and R <= qr:
        ApplySet(v, L, R, x); return
    Push(v, L, R)
    M = (L + R) / 2
    Assign(left(v), L, M, ql, qr, x)
    Assign(right(v), M, R, ql, qr, x)
    sum[v] = sum[left(v)] + sum[right(v)]
```

A range query has the same three overlap cases. Before recursing on a partial overlap it calls `Push`; a fully covered node returns `sum[v]` without descending.

Theorem 6.5 (Lazy-tree correctness)**.** *Range assignment and range sum are correct, preserve the lazy invariant, and each take* $\mathcal{O}(\log n)$ *time.*

Proof. `ApplySet` computes the exact sum of a constant interval and records the pending logical state. `Push` applies precisely the same assignment to the two disjoint children and clears the parent tag; therefore it changes no logical array value or parent sum. On partial overlap, pushing makes both child summaries current, recursive calls are correct by structural induction, and recombination restores the parent sum. The query proof is analogous.

An interval intersects at most two partially covered nodes per level; all fully covered nodes terminate recursion. Therefore only $\mathcal{O}(\log n)$ boundary nodes and their canonical siblings are visited. Each visit performs constant work. □

Assignment tags compose by replacement: a newer assignment erases an older one. For range addition, tags compose by addition and a node sum increases by $x|I_v|$. If both operations coexist, composition order must be specified explicitly.

6.4 Range counting by fractional cascading

EXAMINATION SYLLABUS: ITEM 41

Problem. For a static array, answer

$$C(l, r, x) = |\{i \in [l, r) : a[i] \le x\}| .$$

A merge-sort tree stores at every segment-tree node v the sorted list S_v of values in its interval. A range decomposes into $\mathcal{O}(\log n)$ canonical nodes; binary searching x in each list gives $\mathcal{O}(\log^2 n)$ time.

The specialized **fractional cascading** augmentation eliminates repeated binary searches. When merging the child lists S_L, S_R into S_v, store for every prefix length $p \in [0, |S_v|]$

$$toL_v[p] = \#\{\text{first } p \text{ merged elements originating in } S_L\},$$
$$toR_v[p] = p - toL_v[p].$$

One root binary search computes $p = \text{upper\_bound}(S_{root}, x)$. When traversal enters the left or right child, replace p by `toL[p]` or `toR[p]`. At a fully covered node, add p to the answer.

Theorem 6.6 (Cascaded range counting)**.** *The augmented merge-sort tree uses $\Theta(n \log n)$ build time and memory and answers $C(l, r, x)$ in $\Theta(\log n)$ time.*

Proof. By merge construction, the first p elements of S_v are exactly those at most x when $p = \text{upper\_bound}(S_v, x)$. The bridge values count how many of those elements came from each child, so they equal the corresponding upper-bound positions there. Induction down the query traversal proves every propagated p is correct. Fully covered node intervals are disjoint and cover $[l, r)$, so their counts sum to the answer.

Each array element appears in one node list per level, giving $\Theta(n \log n)$ storage and merge work. The root search costs $\Theta(\log n)$; thereafter each of the $\mathcal{O}(\log n)$ nodes in the ordinary interval traversal uses constant-time bridges. □

The general fractional-cascading method stores sampled copies between related catalogues. The full merge-tree bridges above are a particularly transparent instance tailored to nested segment-tree catalogues.

6.5 Dynamic segment trees

EXAMINATION SYLLABUS: ITEM 42

Suppose indices lie in a huge universe $[0, U)$, perhaps $U = 2^{60}$, while only q points are ever updated. A dynamic segment tree represents the same binary interval hierarchy but allocates a node only when an update enters it. A missing node denotes an all-identity interval.

Listing 6.4: Dynamic point addition

```
1 procedure Add(node reference v, L, R, position, delta):
2     if v is null: v = new zero-initialized node
3     if R - L == 1:
4         v.sum = v.sum + delta; return
```

```
5     M = L + (R - L) / 2
6     if position < M: Add(v.left, L, M, position, delta)
7     else:            Add(v.right, M, R, position, delta)
8     v.sum = Sum(v.left) + Sum(v.right) // Sum(null) = 0
```

Theorem 6.7. *After q point updates, a dynamic segment tree answers range sums correctly in $\mathcal{O}(\log U)$ time per operation and uses $\mathcal{O}(q \log U)$ nodes.*

Proof. Treat every missing subtree as the fully materialized zero tree it abbreviates. Then the update and query recurrences are identical to an ordinary segment tree, so the previous correctness proof applies. Interval length halves at each step, giving depth $\lceil \log_2 U \rceil$. One update creates at most one node per level; nodes shared by update paths only reduce the total. □

This bound can be much smaller than U, though coordinate compression is often better when all relevant coordinates are known offline.

6.6 Coordinate compression

EXAMINATION SYLLABUS: ITEM 43

Given relevant coordinates $x_1, \ldots, x_m$, sort them and remove duplicates to obtain

$$c_0 < c_1 < \cdots < c_{s-1}.$$

Replace each coordinate $x = c_j$ by rank j, found by `lowerBound`.

Theorem 6.8 (Order preservation)**.** *For relevant coordinates x, y,*

$$x < y \iff rank(x) < rank(y), \qquad x = y \iff rank(x) = rank(y).$$

Therefore any algorithm depending only on order or equality may operate on ranks.

Proof. The unique list is strictly increasing. Position in a strictly increasing list is itself a strictly order-preserving bijection between the relevant values and $[0, s)$. □

Sorting costs $\mathcal{O}(m \log m)$, deduplication $\mathcal{O}(m)$, and each online mapping $\mathcal{O}(\log m)$ by binary search. For a threshold x not necessarily present, `lowerBound(c,x)` counts compressed coordinates below x, while `upperBound(c,x)` counts those at most x.

Compression does not preserve distances: ranks of 10 and 10^9 may differ by one. For interval-length calculations, retain original coordinate gaps or use elementary intervals between consecutive endpoints.

6.7 Persistent segment trees

Examination syllabus: item 44

A data structure is **persistent** if updates create new versions while old versions remain queryable. For a segment tree, use **path copying**: clone only nodes on the updated root-to-leaf path and share every untouched subtree.

Listing 6.5: Persistent point addition

```
function PersistentAdd(old, L, R, position, delta):
    new = clone(old)                    // old is never modified
    if R - L == 1:
        new.sum = old.sum + delta
        return new
    M = (L + R) / 2
    if position < M:
        new.left = PersistentAdd(old.left, L, M, position, delta)
    else:
        new.right = PersistentAdd(old.right, M, R, position, delta)
    new.sum = new.left.sum + new.right.sum
    return new
```

Keep one root pointer per version. Nodes are immutable after publication.

Theorem 6.9 (Persistence by path copying)**.** *Each version root represents the array after its version's updates; creating a point-update version takes* $\Theta(\log n)$ *time and new nodes, while queries in any version take* $\Theta(\log n)$ *time.*

Proof. Induct down the copied path. At a leaf, the new value is correct and the old leaf is unchanged. At an internal node, one child pointer refers to the recursively correct new child and the other to an unchanged old subtree whose values did not change; recomputation gives the correct sum. No old node is written, so every old root still reaches exactly its former immutable graph. One node is cloned per tree level, and ordinary query descent has the same depth. □

Reference counting or tracing garbage collection is needed if versions may be discarded. Pointer sharing is safe precisely because shared nodes are immutable.

6.8 Counting values at most x on a subarray

Examination syllabus: item 45

Compress the array values to ranks $0, \dots, \sigma - 1$. Build prefix versions $root_t$ for $t = 0, \dots, n$: $root_0$ stores all zero frequencies, and $root_{t+1}$ adds one at rank $rank(a[t])$. Therefore

$$root_t \text{ stores frequencies of } a[0..t).$$

Let $q = \text{upper\_bound}(c, x)$, the number of distinct compressed values at most x. Then

$$C(l, r, x) = \mathrm{sum}(root_r, [0, q)) - \mathrm{sum}(root_l, [0, q)).$$

Theorem 6.10. *The prefix-persistent construction answers range-counting queries in $\Theta(\log \sigma)$ time after $\Theta(n \log \sigma)$ build time and memory.*

Proof. Induction on t proves the prefix-frequency invariant. Subtracting the frequency vector of prefix $[0, l)$ from that of $[0, r)$ leaves exactly the frequency vector of $[l, r)$. Summing ranks below q counts precisely values at most x. Each persistent addition and each range sum follows one or two paths of height $\Theta(\log \sigma)$. □

This solution and the fractional-cascading merge-sort tree have the same $\Theta(n \log n)$ order of storage. Persistence is particularly convenient when many queries use arbitrary subarrays and the value universe is compressed.

6.9 A subarray order statistic by parallel descent

EXAMINATION SYLLABUS: ITEM 46

To find the zero-based k-th smallest value in $a[l..r)$, descend simultaneously through roots $R = root_r$ and $L = root_l$. At each value interval,

$$leftCount = R.left.sum - L.left.sum$$

is the number of query elements whose rank lies in the left half.

Listing 6.6: Parallel descent for the kth smallest value

```
function Kth(L, R, valueLeft, valueRight, k):
    if valueRight - valueLeft == 1:
        return valueLeft
    leftCount = R.left.sum - L.left.sum
    middle = (valueLeft + valueRight) / 2
    if k < leftCount:
        return Kth(L.left, R.left, valueLeft, middle, k)
    else:
        return Kth(L.right, R.right, middle, valueRight,
                   k - leftCount)
```

Require $0 \le k < r - l$; map the returned rank back to its original value.

Theorem 6.11. *Parallel descent returns the k-th smallest subarray value in $\Theta(\log \sigma)$ time.*

Proof. At every node pair, subtracting prefix frequencies gives exact query frequencies over that value interval. If $k < leftCount$, the desired occurrence lies in the left half with unchanged rank. Otherwise all $leftCount$ left-half occurrences precede it, so it lies right with local rank $k - leftCount$. This is precisely the selection rank recurrence. The value interval halves until one rank remains. □

6.10 Fenwick trees

Examination syllabus: item 47

A Fenwick tree (binary indexed tree) supports point addition and prefix sums in a compact $n+1$-cell array. Use one-based indices and define

$$lowbit(i) = i \,\&\, (-i),$$

the largest power of two dividing i. Store

$$bit[i] = \sum_{j=i-lowbit(i)+1}^{i} a[j].$$

Listing 6.7: Fenwick point addition and prefix sum

```
void add(int i, long long delta) {      // 1 <= i <= n
    for (; i <= n; i += i & -i)
        bit[i] += delta;
}

long long prefixSum(int i) const {      // sum on [1, i]
    long long answer = 0;
    for (; i > 0; i -= i & -i)
        answer += bit[i];
    return answer;
}

long long rangeSum(int l, int r) const { // inclusive [l, r]
    return prefixSum(r) - prefixSum(l - 1);
}
```

Lemma 6.12 (Fenwick decomposition)**.** *Repeatedly replacing i by $i - lowbit(i)$ partitions $[1, i]$ into the disjoint stored intervals encountered by* `prefixSum`*.*

Proof. The first interval ends at i and begins at $i - lowbit(i) + 1$. The next index is exactly one before this beginning, so subsequent intervals are adjacent and disjoint. Clearing the least significant set bit strictly decreases i until zero, at which point the union begins at 1. □

Theorem 6.13 (Fenwick correctness and complexity)**.** *Point addition and prefix/range sums are correct and take $\Theta(\log n)$ worst case, using $\Theta(n)$ memory.*

Proof. The decomposition lemma proves prefix-query correctness from the storage invariant. The indices visited by update are exactly the Fenwick intervals that contain the changed position: adding `lowbit` moves to the next such ancestor. Thus adding δ to each preserves every stored interval sum.

Query clears one set bit per iteration, at most $\lfloor \log n \rfloor + 1$. Update moves through Fenwick ancestors and also has at most this many iterations. Range sum is the difference of two prefixes. □

Fenwick construction can be $\Theta(n \log n)$ by repeated adds or $\Theta(n)$: initialize `bit[i] += a[i]` and add `bit[i]` once to $j = i + lowbit(i)$ when $j \le n$.

6.11 Multidimensional Fenwick trees

EXAMINATION SYLLABUS: ITEM 48

In two dimensions, store sums of rectangles

$$(i - lowbit(i), i] \times (j - lowbit(j), j].$$

Point addition nests two upward loops; prefix query nests two downward loops.

Listing 6.8: Two-dimensional Fenwick operations

```
procedure Add(x, y, delta):
    for i = x; i <= n; i = i + lowbit(i):
        for j = y; j <= m; j = j + lowbit(j):
            bit[i][j] = bit[i][j] + delta

function Prefix(x, y):                  // [1,x] x [1,y]
    answer = 0
    for i = x; i > 0; i = i - lowbit(i):
        for j = y; j > 0; j = j - lowbit(j):
            answer = answer + bit[i][j]
    return answer
```

The Cartesian product of the one-dimensional disjoint decompositions partitions the prefix rectangle, proving query correctness. Update visits exactly every stored rectangle containing the point. Thus a d-dimensional dense Fenwick tree uses $\Theta(\prod_{j=1}^{d} n_j)$ memory and supports operations in

$$\Theta\left(\prod_{j=1}^{d} \log n_j\right)$$

time. An arbitrary axis-aligned box is obtained from 2^d prefix sums by inclusion-exclusion; for fixed d, this does not alter the logarithmic order. The exponential dependence on dimension is the usual curse of dimensionality.

6.12 Forward and reverse Fenwick trees for range maxima

EXAMINATION SYLLABUS: ITEM 49

Assume point values only *increase*. A single maximum Fenwick tree can maintain prefix maxima, but maximum has no inverse, so two prefix answers cannot be subtracted to obtain an arbitrary interval. The remedy is to keep both a forward and a reverse Fenwick tree. Pad the array to a power-of-two length N.

The forward tree stores blocks ending at their index:

$$left[i] = \max a[i - lowbit(i) + 1..i].$$

The reverse tree is an ordinary Fenwick tree on the reflected array $a[N], a[N-1], \ldots, a[1]$. In original coordinates it stores

$$right[i] = \max a[i..i + span(i) - 1], \qquad span(i) = \begin{cases} N, & i = 1, \\ lowbit(i-1), & i > 1. \end{cases}$$

Together these arrays contain the aggregates of all nodes of the complete segment tree: `left` supplies canonical blocks ending at a boundary, while `right` supplies their reflected counterparts starting at a boundary.

Listing 6.9: Range maximum from forward and reverse Fenwick blocks

```
function RangeMax(l, r):                // inclusive, 1-based
    answer = minus infinity
    while r >= l:
        length = r - l + 1
        leftLength = N if l == 1 else lowbit(l - 1)
        rightLength = lowbit(r)
        if leftLength <= length:
            answer = max(answer, right[l])
            l = l + leftLength
        else:
            answer = max(answer, left[r])
            r = r - rightLength
    return answer
```

Theorem 6.14. *Under point increases, the paired structure answers arbitrary range maxima correctly and performs both update and query in $\mathcal{O}(\log n)$ time using $\Theta(n)$ memory.*

Proof. Every queried block begins at the current l or ends at the current r and lies inside the remaining interval. Consuming it therefore maintains the invariant that the chosen blocks are disjoint and their union with the remaining interval is the original query.

At least one of the two candidate blocks always fits. Put $a = l - 1$, $b = r$, and $d = b - a$, the remaining length. For $a > 0$, the left candidate length is $lowbit(a)$; for $a = 0$, regard it as N. If both candidate lengths exceeded d, then $q = \min\{span(l), lowbit(r)\} > d$ would be a power of two dividing both a and b. It would divide $b - a = d$, impossible for $0 < d < q$. Hence the `else` block is valid whenever the left block does not fit. Taking maxima of the disjoint cover proves query correctness.

After a left-block step, $a \leftarrow a + lowbit(a)$, so the least significant set bit of a moves strictly left; this happens at most $\mathcal{O}(\log N)$ times. After a right-block step, $r \leftarrow r - lowbit(r)$, which clears one set bit; there are at most $\mathcal{O}(\log N)$ such steps. Thus the query is logarithmic.

For an increase at position i, perform the usual max-Fenwick update in the forward tree and the same update at reflected position $N - i + 1$ in the reverse tree. Each visits $\mathcal{O}(\log N)$ ancestors. Because values never decrease, a stored maximum can only increase, so replacing every visited aggregate by its maximum with the new value is correct. □

An arbitrary decrease may invalidate a stored maximum without revealing the new one, so the simple monotone update is incorrect for decreases. A segment tree supports general assignments. The paired construction is also called a counter-Fenwick or forward/reverse Fenwick RMQ structure.

6.13 A Fenwick tree of Fenwick trees

EXAMINATION SYLLABUS: ITEM 50

For sparse offline two-dimensional points, a dense $n \times m$ array is wasteful. Build an outer Fenwick tree over compressed x. Each outer cell i contains an inner Fenwick tree over only the y-coordinates that may update that cell.

Preprocessing. For every possible update point (x, y), walk $i = x, i + lowbit(i), \ldots$ and append y to the coordinate list of outer cell i. Sort and deduplicate every list, then allocate its inner Fenwick array.

Operations. A point addition repeats the outer upward walk and updates y's compressed position in each inner tree. A prefix rectangle query repeats the outer downward walk and asks every visited inner tree for the prefix through `upperBound(list[i],y)`.

Theorem 6.15 (Sparse two-dimensional Fenwick bounds)**.** *For q known update coordinates with n_x distinct x-ranks, preprocessing uses $\mathcal{O}(q \log n_x)$ stored inner coordinates. Point update and prefix rectangle sum take $\mathcal{O}(\log n_x \log q)$ time; with comparable coordinate ranges this is $\mathcal{O}(\log^2 q)$.*

Proof. Each point is inserted into one inner coordinate list for every outer Fenwick ancestor, $\mathcal{O}(\log n_x)$ lists, proving storage before and after deduplication. For an update, those are exactly the outer rectangles containing its x; within each, the inner update visits exactly the y-blocks containing its y. For a query, outer Fenwick intervals partition the x-prefix and each inner query partitions the y-prefix. Their Cartesian products are disjoint and cover the rectangle, proving correctness. There are $\mathcal{O}(\log n_x)$ outer visits and each inner binary search/update costs $\mathcal{O}(\log q)$. □

Arbitrary rectangles follow from four prefix rectangles. The method is offline with respect to possible update coordinates; genuinely new coordinates require dynamic inner trees or rebuilding.

Part IV

Search Trees

LECTURE UNIT 7

Binary Search Trees and AVL Trees

7.1 Binary search trees

EXAMINATION SYLLABUS: ITEM 51

Definition 7.1 (Binary-search-tree invariant)**.** A binary tree is a BST if every node v with key $k(v)$ satisfies

$$k(u) < k(v) \quad \text{for all } u \text{ in its left subtree,}$$
$$k(v) < k(u) \quad \text{for all } u \text{ in its right subtree.}$$

We first assume distinct keys. A dictionary with duplicates must declare a policy, such as storing a multiplicity counter in one node.

Lemma 7.2 (Inorder characterization)**.** *A binary tree with distinct keys is a BST if and only if its inorder traversal is strictly increasing.*

Proof. If the invariant holds, structural induction says the left traversal is increasing and below the root, followed by the root, followed by an increasing right traversal above it. Conversely, every node's left-subtree keys appear before it and its right-subtree keys after it in the increasing inorder sequence, giving the BST inequalities. □

7.1.1 Find, insert, and erase

To find x, compare it with the current key: return on equality, go left if smaller and right if larger. A null pointer means absence. To insert, follow the same path to its null endpoint and place a new leaf there.

Theorem 7.3 (Search and insertion correctness)**.** *BST search returns the unique node of key x, if present. Leaf insertion of an absent key preserves the BST invariant. Both take $\Theta(d+1)$ time where d is the reached depth, hence $\mathcal{O}(h)$ for tree height h.*

Proof. If $x < k(v)$, every right-subtree key and v's key exceed x, so only the left subtree can contain it; the symmetric statement holds for $x > k(v)$. Thus discarded subtrees cannot contain the target. At a null child, all ancestors' inequalities specify precisely the interval of legal keys for that position, and the followed comparisons show x lies in it. A new leaf has no descendants, so attaching it there preserves every ancestor inequality. One node is inspected per level. □

To erase node v:

1. If v has no left child, replace its parent link by $v.right$.
2. Else if it has no right child, replace the link by $v.left$.
3. Otherwise let s be the minimum node in $v.right$. Copy s's record into v, then erase s, which has no left child.

Theorem 7.4 (BST deletion correctness)**.** *The deletion procedure removes exactly the requested key, preserves the BST invariant, and takes $\mathcal{O}(h)$ time.*

Proof. A single-child replacement is safe because every key in that child subtree already lies within all intervals imposed by v's ancestors. In the two-child case, the right-subtree minimum s has no left child. It exceeds every left-subtree key and is no greater than every other right-subtree key, so replacing v's key by s's key preserves order around v. Splicing s with its right child is the already proved zero/one-child case. Search plus a descent to the successor traverses at most a constant number of root-to-leaf paths. □

An unbalanced BST may be a chain of height $n - 1$, making every operation linear. Balance mechanisms control this height.

7.1.2 Order-based queries

BST order supports `lowerBound(x)`, predecessor, successor, and reporting all keys in an interval in $\mathcal{O}(h + k)$ time for k reported keys. During a lower bound search, whenever the current key is at least x, save it as the best candidate and go left; otherwise go right. The discarded-side argument proves correctness.

Augment every node by subtree size

$$size(v) = 1 + size(v.left) + size(v.right).$$

Then the key of zero-based rank k is found by comparing k with $s = size(v.left)$: go left if $k < s$, return v if $k = s$, or go right with $k \leftarrow k - s - 1$. The proof is the same rank partition used in persistent order-statistic descent. Updating sizes on a mutation path costs $\mathcal{O}(h)$.

7.1.3 Optional split and merge

`Split(T,x)` returns A, B containing keys $< x$ and $\geq x$. Recursively: if $k(root) < x$, split the right subtree into C, B, set $root.right = C$, and return $root, B$; otherwise symmetrically split the left subtree. Exactly one path is followed, so time is $\mathcal{O}(h)$, and induction proves the key partition.

If every key of A is below every key of B, `Merge(A,B)` may detach the minimum node of B and make A and the remaining B its children. This preserves inorder order and takes $\mathcal{O}(h_A + h_B)$ without a balancing scheme. Treaps and splay trees support more structured logarithmic merge/split operations later in the book.

7.2 AVL trees and their height

EXAMINATION SYLLABUS: ITEM 52

Definition 7.5 (AVL tree)**.** An AVL tree is a BST in which every node v has balance factor

$$bf(v) = height(v.left) - height(v.right) \in \{-1, 0, 1\}.$$

Take $height(null) = 0$ and a leaf's height as 1. Store height at each node.

Theorem 7.6 (AVL height bound)**.** *An AVL tree with n nodes has height $\mathcal{O}(\log n)$; more precisely,*

$$n \geq F_{h+2} - 1,$$

where h is height and $F_0 = 0, F_1 = 1, F_{j+2} = F_{j+1} + F_j$.

Proof. Let $N(h)$ be the minimum nodes in an AVL tree of height h. Then

$$N(0) = 0, \quad N(1) = 1, \quad N(h) = 1 + N(h-1) + N(h-2).$$

Indeed, to have height h, one child has height $h-1$; AVL balance forces the other to have height at least $h-2$, and minimal subtrees attain equality. Induction using the Fibonacci recurrence gives $N(h) = F_{h+2} - 1$.

For $j \geq 2$, $F_j \geq \varphi^{j-2}$, where $\varphi = (1+\sqrt{5})/2$: this follows by induction from $\varphi^2 = \varphi + 1$. Hence

$$n + 1 \geq F_{h+2} \geq \varphi^h, \qquad h \leq \log_\varphi(n+1) = \mathcal{O}(\log n).$$

□

This theorem converts any $\mathcal{O}(h)$ BST operation into $\mathcal{O}(\log n)$ once AVL balance is preserved.

7.3 Rotations and elimination of AVL imbalance

Examination syllabus: item 53

A rotation changes local shape without changing inorder order. A right rotation at y, with $x = y.left$ and $B = x.right$, transforms

$$((A, x, B), y, C) \quad \longmapsto \quad (A, x, (B, y, C)).$$

The displayed inorder sequence is A, x, B, y, C on both sides.

Listing 7.1: AVL rotations and generic rebalancing

```
function RotateRight(y):
    x = y.left; B = x.right
    x.right = y; y.left = B
    UpdateHeight(y); UpdateHeight(x)
    return x

function RotateLeft(x):                // exact mirror image
    y = x.right; B = y.left
```

```
    y.left = x; x.right = B
    UpdateHeight(x); UpdateHeight(y)
    return y

function Rebalance(v):
    UpdateHeight(v)
    if Balance(v) == 2:
        if Balance(v.left) < 0:
            v.left = RotateLeft(v.left) // left-right case
        return RotateRight(v)           // left-left case afterward
    if Balance(v) == -2:
        if Balance(v.right) > 0:
            v.right = RotateRight(v.right) // right-left case
        return RotateLeft(v)
    return v
```

Lemma 7.7 (Rotation correctness)**.** *A left or right rotation preserves the BST key set and inorder order. After bottom-up height recomputation, all stored heights in the rotated fragment are correct.*

Proof. The subtree variables satisfy $A < x < B < y < C$, so both shapes satisfy the same BST inequalities and contain the same nodes. Only the two displayed node heights can change; updating the lower one first and then the new root computes both from already correct child heights. □

Theorem 7.8 (Local AVL repair)**.** *Suppose both child subtrees of v are AVL and $|bf(v)| \le 2$. Then `Rebalance(v)` returns an AVL subtree with the same inorder sequence.*

Proof. Only $bf = \pm 2$ needs work. Suppose $bf(v) = 2$; the mirror case is symmetric. Let $x = v.left$. If $bf(x) \ge 0$, the excess height lies in $x.left$, and one right rotation redistributes heights so both resulting balance factors lie in $[-1, 1]$. Directly write the child heights as $h, h-1$ (or h, h in the deletion equality case) and verify the maxima after rotation.

If $bf(x) = -1$, the tall path bends through $x.right$. A left rotation at x converts it to the preceding outer case; a right rotation at v then restores balance. The middle grandchild's two heights differ by at most one, and checking the three possible balance factors $-1, 0, 1$ gives all new factors in range. Rotation correctness preserves BST order. □

7.4 AVL find, insertion, and deletion

EXAMINATION SYLLABUS: ITEM 54

Search is ordinary BST search. Recursive insertion and deletion perform the BST mutation, then call `Rebalance` while returning through every ancestor.

Listing 7.2: AVL insertion and deletion skeleton

```
function Insert(v, x):
    if v is null: return new Node(x, height = 1)
    if x < v.key:      v.left  = Insert(v.left, x)
    else if v.key < x: v.right = Insert(v.right, x)
    else: return v                        // set semantics
```

```
    return Rebalance(v)

function Erase(v, x):
    if v is null: return null
    if x < v.key:      v.left  = Erase(v.left, x)
    else if v.key < x: v.right = Erase(v.right, x)
    else:
        if v.left is null:  return v.right
        if v.right is null: return v.left
        s = Minimum(v.right)
        v.key = s.key
        v.right = Erase(v.right, s.key)
    return Rebalance(v)
```

In production code, preserve values associated with keys, free erased nodes, and use parent pointers or returned roots consistently.

Theorem 7.9 (AVL operation correctness and complexity)**.** *The algorithms implement set search, insertion, and deletion, preserve both BST and AVL invariants, and take* $\Theta(\log n)$ *worst-case time. Recursive code uses* $\Theta(\log n)$ *stack words.*

Proof. Ordinary BST reasoning proves the search path and the underlying mutation. Before returning from a recursive call, only one child of the current node may have changed height; it is already AVL by induction. Its height changes by at most one, so the current balance magnitude is at most two. The local repair theorem restores AVL balance and correct height without changing inorder order. Induction up to the root proves both invariants globally.

The initial height is $\mathcal{O}(\log n)$. Each level performs one comparison and constant work; a single or double rotation is constant time. Insertion follows one path and deletion follows that path plus a successor path still contained below it. Rebalancing touches at most every ancestor, hence total time and stack depth are $\mathcal{O}(\log n)$. A height-$\Theta(\log n)$ AVL tree gives the matching worst-case order. □

After insertion, one first unbalanced ancestor's rotation restores its old height, so higher rotations are unnecessary, though updating metadata is still required. After deletion, a repaired subtree can become shorter; imbalance may therefore propagate to the root, which is why generic code rebalances every ancestor.

LECTURE UNIT 8

Splay Trees and Implicit Keys

8.1 Splay trees and practical significance

Examination syllabus: item 55

Definition 8.1 (Splay tree)**.** A splay tree is a binary search tree with no explicit balance field. After a node is accessed, a sequence of rotations called **splaying** moves it to the root. An unsuccessful search splays the last visited node.

A single operation can take $\Theta(n)$, but every sufficiently long operation sequence has logarithmic amortized cost. The structure has several practical advantages:

- nodes store no heights, colors, or random priorities;
- frequently or recently accessed keys migrate near the root, exploiting temporal locality;
- split and merge follow directly from splaying;
- the same rotations support implicit sequences and range aggregates.

The tradeoff is latency: amortized bounds do not prevent an individual linear-time operation. Splay trees are unsuitable when strict real-time per-operation bounds are required. Their stronger working-set, static-optimality, and sequential-access properties explain their importance beyond the basic $\mathcal{O}(\log n)$ theorem.

8.2 Zig, zig-zig, zig-zag, and splay

Examination syllabus: item 56

Let x be the accessed node, p its parent, and g its grandparent. Rotations always preserve inorder order by the rotation lemma from chapter 7.

Zig.
If p is the root, rotate once at p in the direction that moves x upward.

Zig-zig.
If x and p are both left children or both right children, first rotate at g to move p upward, then at p to move x upward.

Zig-zag.
If one edge turns left and the other right, rotate first at p, then at g; both rotations move x upward.

Listing 8.1: Bottom-up splaying

```
procedure Splay(x):
    while x.parent is not null:
        p = x.parent; g = p.parent
        if g is null:
            RotateEdge(x, p)                  // zig
        else if (x is p.left) == (p is g.left):
            RotateEdge(p, g); RotateEdge(x, p) // zig-zig
        else:
            RotateEdge(x, p); RotateEdge(x, g) // zig-zag
    return x                                   // new root
```

`RotateEdge(child,parent)` performs the unique left/right rotation that makes `child` the parent and updates parent pointers and all augmented fields bottom-up.

Theorem 8.2 (Structural correctness of splay)**.** *Splaying preserves the BST's nodes and inorder sequence, terminates, and makes x the root.*

Proof. Every elementary rotation preserves the node set and inorder order. A zig reduces x's depth by one; either double step reduces it by two. Thus depth strictly decreases to zero after finitely many steps. No key order changes. □

Double rotations, rather than repeatedly using zig, are what provide the amortized logarithmic theorem: they restructure the entire accessed path, not only its top edge.

8.3 The potential method

Examination syllabus: item 57

For a state D, choose a nonnegative potential $\Phi(D)$. If operation i has actual cost c_i and changes D_{i-1} to D_i, define

$$\widehat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

The terms telescope:

Theorem 8.3 (Potential-method principle)**.** *If $\Phi(D) \geq 0$, $\Phi(D_0) = 0$, and every operation has amortized cost at most a_i, then*

$$\sum_{i=1}^{m} c_i \leq \sum_{i=1}^{m} a_i.$$

More generally, the right side gains $\Phi(D_0)$.

Proof. Rearranging the definition and summing,

$$\sum c_i = \sum \widehat{c}_i + \Phi(D_0) - \Phi(D_m) \leq \sum a_i + \Phi(D_0),$$

because final potential is nonnegative. □

Potential is stored work, not physical time. It may rise on a cheap operation, making its amortized cost larger, and fall on an expensive operation, paying for rotations.

8.4 The access lemma for splaying

EXAMINATION SYLLABUS: ITEM 58

Assign every node a fixed positive weight $w(v)$. For the current tree define

$$s(v) = \sum_{u \text{ in subtree}(v)} w(u), \qquad r(v) = \log_2 s(v), \qquad \Phi = \sum_{v} r(v).$$

Use the number of rotations as actual cost. Write primed quantities after one splay step.

Lemma 8.4 (Log-sum inequality)**.** *If $a, b > 0$ and $a + b \le c$, then*

$$\log_2 a + \log_2 b \le 2 \log_2 c - 2.$$

Proof. By arithmetic-geometric mean, $ab \le ((a+b)/2)^2 \le (c/2)^2$. Take base-two logarithms. □

Theorem 8.5 (Access lemma)**.** *If x is splayed to the root, its amortized number of rotations is at most*

$$3\big(r(root_{\text{before}}) - r(x_{\text{before}})\big) + 1.$$

Proof. We prove a bound for each step in terms of the increase $r'(x) - r(x)$, then telescope.

For a zig, only x, p change rank and $s'(x) = s(p)$, so

$$1 + \Delta\Phi = 1 + r'(p) + r'(x) - r(p) - r(x) = 1 + r'(p) - r(x) \le 1 + r'(x) - r(x).$$

For zig-zag, only x, p, g change rank and $r'(x) = r(g)$. Also $r(p) \ge r(x)$. The new subtrees rooted at p and g are disjoint inside the new subtree of x; the log-sum lemma gives $r'(p) + r'(g) \le 2r'(x) - 2$. Therefore

$$\begin{aligned} 2 + \Delta\Phi &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\le 2 + 2r'(x) - 2 - 2r(x) \le 3(r'(x) - r(x)). \end{aligned}$$

For zig-zig, canceling $r'(x) = r(g)$ and using $r'(p) \le r'(x)$, $r(p) \ge r(x)$, gives

$$2 + \Delta\Phi \le 2 + r'(x) + r'(g) - 2r(x).$$

The *old* subtree of x and the *new* subtree of g are disjoint inside the new subtree of x. Hence the log-sum lemma yields $r(x) + r'(g) \le 2r'(x) - 2$. Substitution gives

$$2 + \Delta\Phi \le 3(r'(x) - r(x)).$$

Sum over all steps. Rank increases of x telescope. The final subtree of x is the entire initial root subtree, so its final rank is the initial root rank. There is at most one zig, contributing the additional 1. □

With uniform weights $w(v) = 1$, $s(root) = n$ and $s(x) \ge 1$. Thus a splay has amortized at most $3\log_2 n + 1$ rotations. For any fixed-size tree, a sequence of m splays takes $\mathcal{O}(m \log n + n \log n)$ actual rotations from an arbitrary initial tree; with a normalized starting potential or a sufficiently long sequence, this is $\mathcal{O}(\log n)$ amortized per access.

8.5 Find, insert, and erase in a splay tree

Examination syllabus: item 59

Find. Perform ordinary BST search. If found, splay the node; otherwise splay the last visited node. The returned root is often assigned back to the tree even when the key is absent.

Insert. Search for x. Under set semantics, if found, merely splay it. Otherwise add a leaf at the search endpoint and splay that leaf to the root.

Erase. Find and splay x. If absent, stop. Detach its left and right subtrees L, R. If L is empty, the new root is R. Otherwise splay the maximum node of L to the root of L; it has no right child, so attach R there.

Theorem 8.6 (Splay dictionary correctness)**.** *The operations implement the ordered-set semantics and preserve the BST invariant. Their amortized time is $\mathcal{O}(\log n)$, while a single operation may take $\Theta(n)$.*

Proof. Search and leaf insertion are correct by the BST theorem, and splaying preserves inorder order. For deletion, every key of L is below every key of R. After the maximum of L becomes its root, its right child is empty; attaching R therefore preserves the inequalities and removes exactly x.

A search path followed by splaying its last node is charged by the access lemma: the rotations restructure precisely that path, and comparisons are within a constant factor of its length. Insertion adds one leaf and then performs such an access; the potential increase caused by one new singleton and changed ancestor sizes is absorbed in $\mathcal{O}(\log(n+1))$ amortized cost under the standard size-rank potential. Deletion uses at most two splays and constant pointer work; removing one node can only decrease subtree-size ranks outside the detached pieces. Thus each operation has logarithmic amortized cost. A chain shows linear actual cost is possible before it is repaired. □

The access lemma above is the essential mathematical ingredient: it explains why the restructuring work of a long access path is paid for by the resulting decrease in future structural potential.

8.6 Splay merge and split

EXAMINATION SYLLABUS: ITEM 60

Merge. Assume every key of A is below every key of B. If either is empty, return the other. Otherwise find the maximum node of A, splay it to A's root, and set its right child to B.

Split. To split T by x into keys $< x$ and keys $\geq x$, search for the lower bound of x and splay it if it exists. If it becomes root $r \geq x$, detach $r.left$ as A and keep r with its right subtree as B. If no lower bound exists, all keys belong to $A = T$ and B is empty. A last-accessed-node variant handles both boundary cases uniformly.

Theorem 8.7 (Merge/split correctness and time)**.** *Splay merge and split return exactly the specified key partitions, preserve BST order, and cost $\mathcal{O}(\log n)$ amortized.*

Proof. The maximum of A has no right child after it is splayed; since all B keys are larger, attaching B there yields the concatenated inorder sequence. For split, every key in the root's detached left subtree is below the lower bound, and the root plus its right subtree are at least it; no other keys exist. Parent pointers and augmented fields are updated at the cut. Each operation performs one search/splay plus constant pointer work, so the access-lemma bound applies. □

Repeated use of split and merge is safe only if their key preconditions are checked; merging overlapping key ranges silently destroys the BST invariant.

8.7 An implicit-key search tree for a dynamic sequence

EXAMINATION SYLLABUS: ITEM 61

An **implicit-key tree** stores a sequence but no explicit position keys. The inorder order is the sequence order, and a node's zero-based implicit index is

$$size(v.left) + \sum_{\substack{u \text{ ancestor of } v \\ v \text{ lies in } u.right}} (size(u.left) + 1).$$

Store at every node

$$size(v) = 1 + size(v.left) + size(v.right), \qquad sum(v) = value(v) + sum(v.left) + sum(v.right).$$

Every rotation recomputes the lower node and then the new upper node. Searching by rank compares k with $size(left)$, as in an order-statistic BST, and splays the found node. Define `SplitBySize(T,k)` to return the first k elements and the remaining suffix; it splays the node of rank k (with endpoint cases) and detaches its left subtree. `Merge` needs only the sequence-order precondition: all nodes of the first argument precede those of the second.

The required array operations reduce to split and merge:

Operation	Reduction
Insert x at position i	$(A, B) = Split(T, i)$; $T = Merge(Merge(A, x), B)$
Erase position i	split at i, split one element from suffix, discard it, merge the outer pieces
Assign position i	find by rank, splay, change value, recompute aggregate
Sum on $[l, r)$	split at l, then at $r - l$; read middle root's sum; merge back

Theorem 8.8 (Implicit-tree correctness and complexity)**.** *The reductions implement dynamic sequence insertion, deletion, point assignment, and range sum. Using a splay tree, each takes* $\mathcal{O}(\log n)$ *amortized time and* $\Theta(n)$ *storage.*

Proof. Structural induction proves subtree size equals the number of inorder elements and subtree sum equals their sum. Rank descent therefore selects the requested position. Split partitions the inorder sequence after exactly k nodes; merge concatenates two inorder sequences. The table's compositions consequently have the stated sequence effects, and the isolated middle subtree contains exactly the queried range. Augmented fields are recomputed after every rotation, cut, join, and value change, preserving aggregates. A constant number of amortized logarithmic split/merge/access operations gives the bound. □

Lazy tags can extend the structure to range reversal, addition, or assignment. For reversal, store a Boolean flag, swap children when applying it, and push it before any structural descent; the aggregate must transform consistently.

Part V

Hashing and Randomized Dictionaries

LECTURE UNIT 9

Hashing, Filters, and Skip Lists

9.1 Hash functions and random families

EXAMINATION SYLLABUS: ITEM 62

Let U be a universe of keys and $[m] = \{0, \dots, m-1\}$ table indices.

Definition 9.1 (Hash function and collision)**.** A hash function is a map $h : U \to [m]$. Distinct keys $x \neq y$ collide when $h(x) = h(y)$. If $|U| > m$, the pigeonhole principle makes collisions inevitable.

A single fixed public hash function can be defeated by a key set concentrated in one bucket. Probability guarantees therefore choose a function at random from a family $\mathcal{H}$, after the key set is fixed.

Definition 9.2 (Universal family)**.** $\mathcal{H}$ is universal if, for all distinct $x, y \in U$,

$$\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}.$$

Definition 9.3 (k-independence)**.** A family $\mathcal{H} \subseteq \{h : U \to [m]\}$ is k-independent and uniform if for every distinct $x_1, \dots, x_k$ and every $y_1, \dots, y_k \in [m]$,

$$\Pr[h(x_1) = y_1, \dots, h(x_k) = y_k] = m^{-k}.$$

2-independence implies universality, since

$$\Pr[h(x) = h(y)] = \sum_{z \in [m]} \Pr[h(x) = z, h(y) = z] = m \cdot m^{-2} = 1/m.$$

9.1.1 Affine and polynomial examples

Let p be prime and identify keys and table positions with the finite field $\mathbb{F}_p$. Choose a, b independently and uniformly from $\mathbb{F}_p$, and set

$$h_{a,b}(x) = ax + b \pmod{p}.$$

Theorem 9.4 (Strong pairwise independence)**.** *The affine family $\{h_{a,b}\}$ is 2-independent.*

Proof. For distinct x, y and prescribed u, v, the equations

$$ax + b = u, \qquad ay + b = v$$

have the unique solution $a = (u - v)(x - y)^{-1}$, $b = u - ax$, because nonzero $x - y$ has an inverse in a field. Exactly one of the p^2 equally likely coefficient pairs gives the prescribed output pair, so its probability is p^{-2}. □

For k-independence, choose independent uniform coefficients $c_0, \dots, c_{k-1} \in \mathbb{F}_p$ and use

$$h_c(x) = \sum_{j=0}^{k-1} c_j x^j \pmod{p}.$$

Theorem 9.5 (Polynomial hashing)**.** *Degree-below-k polynomial hashing over $\mathbb{F}_p$ is k-independent.*

Proof. For distinct $x_1, \dots, x_k$, prescribing $h(x_i) = y_i$ gives a linear system whose coefficient matrix is Vandermonde. Its determinant is

$$\prod_{1 \le i < j \le k} (x_j - x_i) \neq 0$$

in the field, so exactly one coefficient vector produces the k outputs. Among the p^k uniform coefficient vectors, its probability is p^{-k}. □

For a nonprime table size, one may hash into a larger prime field and use an additional balanced reduction, or use a word-oriented universal family. A casual "take modulo m" can weaken the exact collision bound and should be analyzed, not assumed.

9.2 Separate-chaining hash tables

EXAMINATION SYLLABUS: ITEM 63

A chaining table has m buckets, each a linked list or small dynamic array. Key x belongs to bucket $h(x)$. Let n be the number of stored keys and

$$\alpha = n/m$$

the load factor.

- `find(x)` scans bucket $h(x)$;
- `insert(x)` first checks set semantics, then adds to that bucket;
- `erase(x)` finds and unlinks the key.

Theorem 9.6 (Expected chaining cost)**.** *If h is chosen independently of a fixed key set from a universal family, the expected time for an unsuccessful search is $\mathcal{O}(1 + \alpha)$. The same bound holds for a successful search when the queried stored key is fixed independently of h.*

Proof. For a query x, let X_y indicate collision with stored key $y \neq x$. The number of inspected nonmatching keys is at most $X = \sum_y X_y$. By universality and linearity of expectation,

$$\mathbb{E}[X] = \sum_{y} \mathbb{P}[h(y) = h(x)] \leq \frac{n}{m} = \alpha.$$

The hash calculation, bucket access, and possible matching record add constant work. Independence among collision indicators is unnecessary. □

9.2.1 Rehashing

Keep α between fixed constants, for example grow to $2m$ when $n > m$ and optionally shrink when $n < m/4$. On resize, allocate a new bucket array, choose a fresh random hash function, and insert every existing key according to its new hash. Rehashing is necessary because changing m or h changes bucket locations.

Theorem 9.7 (Amortized expected dictionary time)**.** *With geometric resizing and universal hashing, chaining supports find, insert, and erase in expected $\mathcal{O}(1)$ time when the load factor is bounded by a constant; insert and erase have this bound amortized over resizing.*

Proof. The preceding theorem gives expected constant bucket work. A growth rehash at size n costs $\Theta(n)$, but another growth cannot occur until $\Theta(n)$ insertions; charge constant credits to them. A shrink threshold separated from the grow threshold similarly prevents alternating operations from causing repeated rehashes. Freshly choosing h restores the universality experiment for the fixed current set. □

Worst-case lookup is still $\Theta(n)$ if many keys collide. Expected guarantees depend on keeping the random hash seed secret from an adaptive adversary or using a family strong enough for the adversarial model.

9.3 Static perfect hashing

EXAMINATION SYLLABUS: ITEM 64

Perfect hashing stores a fixed set S of n distinct keys in $\Theta(n)$ space and answers membership in $\Theta(1)$ *worst-case* time. The FKS scheme uses two levels.

Choose a universal first-level hash into $m = n$ buckets. If bucket i receives b_i keys, allocate a second-level table of size b_i^2 and choose a universal hash for that bucket until it has no collision.

Lemma 9.8 (Sum of squared bucket sizes)**.** *For a universal first-level hash with $m = n$,*

$$\mathbb{E}\left[\sum_{i} b_i^2\right] < 2n.$$

Proof. Expand by ordered pairs:

$$\sum_{i} b_i^2 = \sum_{x \in S} \sum_{y \in S} \mathbf{1}[h(x) = h(y)].$$

The n diagonal terms equal one. Each of the $n(n-1)$ ordered distinct pairs collides with probability at most $1/n$. Taking expectations gives a value below $n + n(n-1)/n < 2n$. □

By Markov's inequality, a random first hash satisfies $\sum b_i^2 \le 4n$ with probability at least $1/2$, so retrying needs fewer than two trials in expectation.

Lemma 9.9 (Collision-free second level)**.** *For b keys hashed universally into b^2 cells, the probability of any collision is below $1/2$.*

Proof. There are $\binom{b}{2}$ unordered pairs. By the union bound,

$$\mathbb{P}(\text{some collision}) \le \binom{b}{2}\frac{1}{b^2} < \frac{1}{2}.$$

□

Theorem 9.10 (FKS guarantees)**.** *FKS hashing uses $\Theta(n)$ space, has expected $\Theta(n)$ construction time, and answers membership in $\Theta(1)$ worst-case time.*

Proof. Accepting only first hashes with $\sum b_i^2 \le 4n$ bounds all second-level cells linearly. Each bucket obtains a collision-free second hash after a constant expected number of trials, and trial work is $\Theta(b_i^2 + b_i)$; summing gives $\Theta(n)$ expected work. A query evaluates the first hash, then the selected second hash, and compares the stored key to reject an unrelated empty/fingerprint match: a constant number of RAM operations, independent of bucket size. □

The construction is randomized; once successfully built, query time and correctness are deterministic. Dynamic perfect hashing is more involved.

9.4 Open addressing

EXAMINATION SYLLABUS: ITEM 65

Open addressing stores all records directly in one array. A probe function $h(x, i)$, $i = 0, \dots, m-1$, must visit every cell exactly once for each key. Cells have states `EMPTY`, `OCCUPIED`, or `DELETED`.

- `find` follows the probe sequence until finding x or an `EMPTY` cell. It may not stop at `DELETED`.
- `insert` searches for duplicates and remembers the first reusable `DELETED` cell; it inserts there or at the first `EMPTY` cell.
- `erase` marks the found cell `DELETED`; clearing it to `EMPTY` could break a later key's search path.

These rules are correct because every inserted key remains on its own probe prefix before the first never-used cell. Tombstones preserve that reachability but slow search, so rehash when occupancy or tombstone fraction crosses a threshold. A rehash creates a fresh table and reinserts only occupied records.

9.4.1 Uniform probing and double hashing

Under the **uniform hashing assumption**, every key's probe order is an independent uniform random permutation of the m cells.

Theorem 9.11 (Uniform-probing bounds)**.** *At load $\alpha = n/m < 1$, the expected probes of an unsuccessful search or insertion are at most $1/(1-\alpha)$. The average expected probes of a successful search are at most*

$$\frac{1}{\alpha}\ln\frac{1}{1-\alpha} + \mathcal{O}(1/n).$$

Proof. For an unsuccessful search to make more than j probes, its first j cells must all be occupied. Sampling without replacement gives probability at most $(n/m)^j = \alpha^j$. The tail-sum formula yields

$$\mathbb{E}[X] = \sum_{j\geq 0}\mathbb{P}(X > j) \leq \sum_{j\geq 0}\alpha^j = \frac{1}{1-\alpha}.$$

A key inserted when i cells were occupied has expected insertion/search cost at most $m/(m-i)$, and its later successful search retraces no more probes. Averaging over the n insertion times gives

$$\frac{1}{n}\sum_{i=0}^{n-1}\frac{m}{m-i} = \frac{1}{\alpha}\sum_{j=m-n+1}^{m}\frac{1}{j} \leq \frac{1}{\alpha}\ln\frac{1}{1-\alpha} + \mathcal{O}(1/n),$$

using the integral bound for the harmonic sum. □

Double hashing uses

$$h(x,i) = (h_1(x) + ih_2(x)) \bmod m.$$

If m is prime and $h_2(x) \in \{1,\ldots,m-1\}$, multiplication by the nonzero step permutes all residues, so every probe sequence visits every cell. With independent uniform h_1, h_2, double hashing realizes $m(m-1)$ arithmetic permutations and is analyzed by the uniform-probing theorem as the standard ideal model. It does not generate all $m!$ permutations, so the assumption and hash quality should be stated.

9.4.2 Linear probing

Linear probing uses $h(x,i) = (h(x)+i) \bmod m$. It is cache-friendly but develops **primary clusters**: keys hashing anywhere into a consecutive occupied run extend the same run.

Theorem 9.12 (Linear-probing bounds)**.** *If home positions are fully independent and uniform and alpha $\leq 1-\varepsilon$, then expected probes are $\mathcal{O}(\varepsilon^{-2})$ for an unsuccessful search or insertion and $\mathcal{O}(\varepsilon^{-1})$ for a successful search. In the classical random-occupancy approximation the corresponding means are*

$$\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right), \qquad \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right).$$

Proof of the asymptotic orders. An unsuccessful search scans the occupied run beginning at its home cell. If it scans at least t cells, some interval of length between $t/2$ and $2t$ around that cell received at least as many home hashes as cells; otherwise an empty cell would split the run earlier. For an interval of length s, the number of home hashes is binomial with mean αs. A Chernoff bound gives

$$\mathbb{P}(X \geq s) \leq \exp(-c\varepsilon^2 s)$$

for an absolute $c > 0$. Applying this to dyadic interval lengths and a constant number of alignments yields

$$\mathbb{P}(\text{run length} \geq t) \leq C \exp(-c'\varepsilon^2 t).$$

Summing the tail bounds gives expected unsuccessful cost $\mathcal{O}(\varepsilon^{-2})$. A successful key's displacement is the run length at its insertion load; averaging the run-growth costs from load 0 to $1 - \varepsilon$, or equivalently integrating the unsuccessful bound over the load, gives $\mathcal{O}(\varepsilon^{-1})$. Exact enumeration of runs under the classical model evaluates these sums to the displayed half-formulas. □

Limited-independence hashes require care: five-wise independence suffices for constant expected linear-probing time at any fixed load below one, whereas mere pairwise independence does not. All open-addressing variants keep α bounded away from one by geometric rehashing, which makes their stated bounds constant and pays rebuild cost amortized over $\Theta(n)$ updates.

9.5 Bloom filters

Examination syllabus: item 66

A Bloom filter represents a set approximately in an m-bit array, initially zero, using k hash functions into the bit positions.

- To insert x, set all bits $B[h_j(x)] \leftarrow 1$.
- To query x, return "possibly present" iff all k bits are one.

Theorem 9.13 (Correctness and false-positive probability)**.** *Bloom filters have no false negatives after insertions. Under independent uniform hashing, after n insertions a nonmember's false-positive probability is approximated by*

$$f(k) = \left(1 - e^{-kn/m}\right)^k.$$

The optimal real-valued number of hashes is

$$k^* = \frac{m}{n} \ln 2,$$

rounded to a nearby positive integer.

Proof. Every bit set by an inserted key remains one, so all of that key's queried bits are one. A fixed bit remains zero after kn independent placements with exact probability $(1 - 1/m)^{kn} \approx e^{-kn/m}$. Treating queried bit states as asymptotically independent gives the displayed standard probability; exact finite analysis differs only by small dependence corrections.

Let $a = n/m$ and differentiate

$$g(k) = \ln f(k) = k \ln(1 - e^{-ak}).$$

The unique interior minimum satisfies

$$\ln(1 - e^{-ak}) + \frac{ake^{-ak}}{1 - e^{-ak}} = 0,$$

whose solution is $ak = \ln 2$: both terms become $-\ln 2$ and $+\ln 2$. Endpoint behavior and the derivative sign give the minimum. □

At the optimum, half the bits are expected to remain zero and

$$f(k^*) \approx \exp\left(-\frac{m}{n}(\ln 2)^2\right), \qquad \frac{m}{n} \approx \frac{-\ln f}{(\ln 2)^2}.$$

Bloom filters do not directly support deletion because clearing a shared bit can create false negatives. A counting Bloom filter replaces bits by counters, at a memory cost and with overflow considerations.

9.6 Cuckoo filters

EXAMINATION SYLLABUS: ITEM 67

A cuckoo filter stores a short f-bit fingerprint $fp(x)$, not the full key, in one of two buckets. Each bucket has b slots. With a power-of-two bucket count, define

$$i_1 = h(x), \qquad i_2 = i_1 \oplus h_f(fp(x)).$$

The alternative relation is involutive: $i_1 = i_2 \oplus h_f(fp(x))$. Thus a stored fingerprint can be moved without the original key.

Lookup tests both candidate buckets. Insertion uses a free slot if available; otherwise it evicts a random resident fingerprint, inserts the new one, moves the evicted fingerprint to its alternate bucket, and repeats up to a limit before rehashing. Deletion removes one matching fingerprint.

Theorem 9.14 (Filter invariant and error bound)**.** *If insertion succeeds and no unsafe deletion occurs, a cuckoo filter has no false negatives. With uniform f-bit fingerprints, a nonmember false-positive probability is at most*

$$\frac{2b}{2^f},$$

and more precisely is approximately $1 - (1 - 2^{-f})^{2b}$ when both buckets are full and slots behave independently.

Proof. Maintain the invariant that every stored fingerprint lies in one of the two buckets computed for its key. Initial placement satisfies it. On eviction, the involutive XOR formula computes the resident fingerprint's other legal bucket, so relocation preserves it. Lookup checks both legal locations, proving no false negative for a successfully inserted, unremoved key.

A nonmember query examines at most $2b$ fingerprints. Each equals its uniform query fingerprint with probability 2^{-f}; the union bound gives $2b/2^f$ without needing slot independence. Under independence, the probability no slot matches is $(1 - 2^{-f})^{2b}$. □

False-positive deletion is dangerous: deleting an absent key's coincident fingerprint may remove a present key and create a false negative. Delete only when membership is known externally, or store counts. Achievable load and insertion failure constants depend on bucket size and relocation policy and require a separate random-graph analysis; the invariant and error theorem above do not rely on a particular numerical threshold.

9.7 Skip lists

EXAMINATION SYLLABUS: ITEM 68

A skip list consists of sorted linked levels. Level 0 contains every key. Independently promote each node from level j to $j + 1$ with probability $p \in (0, 1)$. A node height $H \geq 1$ therefore satisfies

$$\mathbb{P}(H \geq k) = p^{k-1}.$$

Search begins at the highest sentinel, moves right while the next key is below the target, then drops one level. Insertion records the last node before the target at each level, samples a height, and splices the new node into those levels. Erasure uses the same predecessor path to unlink the node wherever it appears.

Theorem 9.15 (Skip-list correctness)**.** *Search finds a key exactly when it is present; insertion and erasure preserve the sorted-level and nesting invariants.*

Proof. At each level, moving right stops at the last node below the target. Because every higher-level node also appears below, dropping retains a valid predecessor and cannot skip the target. At level zero, the successor is the least key at least the target. Splicing between recorded predecessor and successor preserves sortedness; placing the new node in every level below its sampled height preserves nesting. Unlinking its occurrences preserves both properties. □

Theorem 9.16 (Expected skip-list bounds)**.** *For n keys, expected storage is $n/(1 - p)$, expected search/update time is*

$$\mathcal{O}\left(\frac{1}{p} \log_{1/p} n\right),$$

and maximum height is $\mathcal{O}(\log n)$ with high probability.

Proof. One node's expected number of levels is

$$\mathbb{E}[H] = \sum_{k \geq 1} \mathbb{P}(H \geq k) = \sum_{j \geq 0} p^j = \frac{1}{1-p},$$

so linearity gives expected total pointers $n/(1-p)$.

For maximum height M, the union bound gives

$$\mathbb{P}(M \geq k) \leq np^{k-1}.$$

Taking $k = c \log_{1/p} n + 1$ makes this at most n^{1-c}.

Analyze a search path backward from level zero. At one level, the number of horizontal nodes encountered before reaching a node promoted to the next level is geometric with success probability p, hence expectation $1/p$. There are $\mathcal{O}(\log_{1/p} n)$ relevant levels in expectation (and with high probability), so linearity of expectation gives the search bound. Insertion and erasure add constant work per visited level plus an expected constant number of splices per node level. □

The worst case is linear—all nodes might receive height one—but its probability decays rapidly. The common choice $p = 1/2$ gives expected $\mathcal{O}(\log n)$ time and about two forward pointers per key.

Part VI

Treaps and External-Memory Search Trees

LECTURE UNIT 10

Treaps, B-Trees, and Red-Black Trees

10.1 Cartesian trees (treaps)

EXAMINATION SYLLABUS: ITEM 69

Definition 10.1 (Treap)**.** Each node has a distinct search key x and a distinct priority p. A min-priority treap simultaneously satisfies

1. the BST invariant by x; and
2. the min-heap invariant by p: every parent priority is below its child priorities.

This is also called the Cartesian tree of the key-priority pairs.

Theorem 10.2 (Uniqueness)**.** *Distinct keys and priorities determine a unique treap.*

Proof. The root must be the node of globally minimum priority by heap order. BST order forces all smaller keys into its left subtree and all larger keys into its right subtree. Apply the same argument recursively to each side. This constructs and uniquely determines the entire tree. □

If priorities are independent continuous random variables, their relative order is a uniform random permutation. The root is a uniformly random key, and conditional on it, the two sides have independent random relative priority orders. Thus the treap has the same distribution as a random BST.

Theorem 10.3 (Expected depth)**.** *Let keys have sorted ranks* $0, \dots, n-1$*. The expected depth of rank* i *in a random-priority treap is*

$$\mathbb{E}[\mathit{depth}(i)] = H_{i+1} + H_{n-i} - 2 = \mathcal{O}(\log n).$$

Proof. For $j < i$, node j is an ancestor of i exactly when its priority is the minimum among ranks $j, j+1, \dots, i$. Necessity follows because their Cartesian subtree root must separate the interval; sufficiency follows from the recursive minimum-priority construction. Among $i-j+1$ random priorities, each is equally likely to be minimum, so the probability is $1/(i-j+1)$. Symmetrically, for $j > i$ it is $1/(j-i+1)$. Depth is the number of proper ancestors, hence by linearity of expectation

$$\mathbb{E}[\mathit{depth}(i)] = \sum_{j=0}^{i-1} \frac{1}{i-j+1} + \sum_{j=i+1}^{n-1} \frac{1}{j-i+1} = H_{i+1} + H_{n-i} - 2.$$

Since $H_k \le 1 + \ln k$, this is $\mathcal{O}(\log n)$. □

Expected depth does not by itself protect against priorities chosen by an adaptive adversary. Priorities must be generated independently of the keys and operations; the expected-depth theorem then applies to every fixed searched rank.

10.1.1 Linear construction from sorted keys

When nodes arrive in increasing key order, their inorder order is already fixed. Maintain a stack equal to the right spine of the treap built so far; its priorities increase from bottom to top.

Listing 10.1: Linear Cartesian-tree construction

```
stack = empty
for node x in increasing key order:
    last = null
    while stack is not empty and stack.top.priority > x.priority:
        last = stack.pop()
    x.left = last
    if stack is not empty:
        stack.top.right = x
    stack.push(x)
root = stack.bottom
```

Theorem 10.4 (Linear-build correctness)**.** *The stack algorithm constructs the unique treap in* $\Theta(n)$ *time and* $\mathcal{O}(n)$ *auxiliary pointer storage.*

Proof. Before inserting x, the stack is the existing tree's right spine with increasing priorities. Popped nodes form a suffix of that spine, all with priority above $p(x)$; the last popped node is the root of exactly the maximal suffix subtree whose keys are below x and priorities exceed $p(x)$, so it becomes $x.left$. The remaining top, if any, is the nearest earlier node with smaller priority and must have x as its right child. These links preserve key inorder and priority heap order, and the new stack is the new right spine. Induction proves the final tree satisfies both invariants, hence uniqueness proves it is the treap.

Every node is pushed once and popped at most once, so total loop work is $\Theta(n)$. □

10.2 Treap merge and split

Examination syllabus: item 70

Merge. Given treaps A, B with every key in A below every key in B, the lower priority root must be the merged root. Recurse only on the adjacent boundary subtree.

Listing 10.2: Treap merge and split

```
function Merge(A, B):
    if A is null: return B
    if B is null: return A
    if A.priority < B.priority:
```

```
        A.right = Merge(A.right, B)
        Update(A); return A
    else:
        B.left = Merge(A, B.left)
        Update(B); return B

function Split(T, x):                    // keys < x, keys >= x
    if T is null: return (null, null)
    if T.key < x:
        (C, B) = Split(T.right, x)
        T.right = C; Update(T)
        return (T, B)
    else:
        (A, C) = Split(T.left, x)
        T.left = C; Update(T)
        return (A, T)
```

Theorem 10.5 (Treap merge/split correctness)**.** *Merge and split preserve both invariants and return exactly the required key sets. Their time is $\mathcal{O}(h)$, where h is the height of the involved treap, hence expected $\mathcal{O}(\log n)$ under independent random priorities.*

Proof. For merge, the minimum-priority root among the union is the smaller of the two roots. If it is $A.root$, its left subtree is unchanged and all remaining keys belong in the merge of $A.right$ and B; the key precondition remains true. The other case is symmetric. Induction plus root priority proves both invariants.

For split, if $T.key < x$, the root and its entire left subtree belong to the first answer; only the right subtree can straddle x. Recursively splitting it and reattaching its below-x part preserves both orders. The case $T.key \geq x$ is symmetric. Every recursive call moves one level down, and each level does constant work. The expected-height theorem gives the expected bound. □

10.3 Treap insertion and deletion through merge/split

Examination syllabus: item 71

For a new key x with independent random priority:

$$(A, B) = Split(T, x), \qquad T = Merge(Merge(A, new(x)), B).$$

Under set semantics, first check absence or use a three-way split to isolate equal keys. To erase x, split into A with keys $< x$, M with keys $= x$, and B with keys $> x$; discard M and set $T = Merge(A, B)$. With integer keys, the second split may use $x + 1$ if overflow is impossible; a comparator-based implementation should instead provide `SplitLE` or an equality-aware split.

Theorem 10.6. *The reductions implement set insertion and deletion while preserving the treap invariants. They take expected $\mathcal{O}(\log n)$ time and $\mathcal{O}(\log n)$ expected recursive stack space.*

Proof. Split creates the exact ordered key intervals. The singleton priority is independent, and merge's preconditions hold at both joins, so merge correctness gives the unique treap on the

union. Deletion discards exactly the equality interval and merges two correctly ordered sides. A constant number of expected logarithmic merge/split calls gives the time; their recursion follows tree paths. □

An alternative recursive insertion descends by key until the new priority beats the current root, then splits the current subtree around the new key. Recursive deletion replaces a found root by `Merge(left,right)`. These are algebraically equivalent.

10.4 B-trees: definition, role, and search

EXAMINATION SYLLABUS: ITEM 72

Binary trees optimize comparisons but use one pointer traversal per level. On disk or across cache misses, moving an entire block is far more expensive than several comparisons inside it. B-trees increase branching so one node fits one page.

Definition 10.7 (B-tree of minimum degree $t \geq 2$)**.** A B-tree satisfies:

1. Each node stores sorted keys $k_1 < \cdots < k_q$.
2. An internal node with q keys has $q + 1$ children whose key ranges are separated by those keys.
3. Every nonroot node has $t - 1 \leq q \leq 2t - 1$ keys.
4. A nonempty root has $1 \leq q \leq 2t - 1$ keys; if internal, it has at least two children.
5. All leaves have the same depth.

A node with $2t - 1$ keys is **full**. Search binary-searches the keys within the current node. On equality it succeeds; otherwise it follows the unique child interval, or fails at a leaf.

Theorem 10.8 (B-tree search correctness)**.** *Search returns the record of key x exactly when present. It reads one node per level, hence uses $\mathcal{O}(h)$ page transfers and $\mathcal{O}(h \log t)$ comparisons with binary search inside nodes.*

Proof. Node separators partition all descendant keys into disjoint intervals. The result of the within-node search either finds equality or identifies the unique interval that can contain x; all other children are excluded. Induction down the path proves correctness. Equal leaf depth bounds the path by $h + 1$ nodes. □

In an external-memory model with block size B words, choose $t = \Theta(B)$. Then search costs $\Theta(\log_B n)$ I/Os, explaining B-trees' use in databases and filesystems. CPU implementations may use linear search inside small nodes for cache and branch efficiency despite its $\mathcal{O}(t)$ comparison bound.

10.5 B-tree height

EXAMINATION SYLLABUS: ITEM 73

Theorem 10.9 (B-tree depth bound)**.** *A nonempty B-tree of minimum degree t, height h (root at depth 0), and n keys satisfies*

$$n \geq 2t^h - 1, \qquad h \leq \log_t \frac{n+1}{2}.$$

Proof. For $h = 0$, the root has at least one key. For $h \geq 1$, the root has at least two children. Every nonroot internal node has at least t children, so depth $d \geq 1$ contains at least $2t^{d-1}$ nodes. Every nonroot node contains at least $t - 1$ keys. Therefore

$$\begin{aligned} n &\geq 1 + (t-1)\sum_{d=1}^{h} 2t^{d-1} \\ &= 1 + 2(t-1)\frac{t^h - 1}{t-1} = 2t^h - 1. \end{aligned}$$

Rearranging proves the height bound. □

Conversely a height-h tree has at most $(2t)^{h+1} - 1$ keys, so height is $\Theta(\log_t n)$ when t is fixed and occupancy stays within the definition's bounds.

10.6 B-tree insertion

EXAMINATION SYLLABUS: ITEM 74

Insertion uses a top-down rule: *never descend into a full child.* Splitting a full node with keys

$$k_1, \ldots, k_{t-1}, k_t, k_{t+1}, \ldots, k_{2t-1}$$

promotes median k_t to the parent and leaves two nodes with $t - 1$ keys each. If the root is full, create a new empty root, make the old root its child, and split that child; height increases by one.

Listing 10.3: B-tree insertion into a nonfull node

```
procedure Insert(T, k):
    r = T.root
    if r is full:
        s = new internal node with child[0] = r
        T.root = s
        SplitChild(s, 0)
        InsertNonFull(s, k)
    else:
        InsertNonFull(r, k)

procedure InsertNonFull(x, k):
    if x is a leaf:
        shift larger keys right and insert k in sorted position
        return
    i = child interval containing k
    if x.child[i] is full:
```

```
17          SplitChild(x, i)
18          if k == x.key[i]: handle duplicate policy
19          if k > x.key[i]: i = i + 1
20      InsertNonFull(x.child[i], k)
```

`SplitChild(parent,i)` allocates the right half, moves the upper $t-1$ keys and, if internal, upper t child pointers into it, promotes the median into the parent, and leaves the lower $t-1$ keys in the original child.

Theorem 10.10 (B-tree insertion correctness and complexity)**.** *Top-down insertion preserves all B-tree invariants and inserts the key in $\mathcal{O}(th)$ RAM time and $\mathcal{O}(h)$ page transfers. With binary search inside nodes, comparisons are $\mathcal{O}(h \log t)$, excluding key shifts.*

Proof. Splitting preserves sorted inorder order: lower keys remain left, the median separates in the parent, and upper keys move right. Each resulting nonroot node has exactly $t-1$ keys and child depths are unchanged. A root split creates one new level above all old leaves, preserving equal leaf depth.

By induction, `InsertNonFull` is called only on a nonfull node. Before it descends, it splits a full child, so the chosen child afterward is nonfull. At a leaf, at most $2t-2$ existing keys shift and one new key is added without overflow. All minimum-occupancy conditions remain true. Only one path is followed, with constant-many $\Theta(t)$ array moves per level and one page visit per level. □

10.7 B-tree deletion

Examination syllabus: item 75

Deletion uses the dual top-down rule: *before descending into a child, ensure it has at least t keys.* Then removing one key below cannot underflow it.

For a node x and target k, apply the following exhaustive cases.

1. If k occurs in a leaf, remove it by shifting later keys left.
2. If $k = x.key[i]$ occurs in an internal node:
   (a) if left child c_i has at least t keys, replace k by its predecessor and recursively delete that predecessor from c_i;
   (b) else if right child c_{i+1} has at least t keys, use the successor symmetrically;
   (c) else both have $t-1$ keys: merge them with separator k into one $2t-1$-key node and recurse there.
3. If k is not in internal node x, let c_i be its possible child. Before descending, if c_i has only $t-1$ keys:
   (a) borrow from an adjacent sibling with at least t keys: move the parent separator into c_i, move the sibling's adjacent key up to the parent, and move the corresponding child pointer if internal;
   (b) otherwise merge c_i, a parent separator, and an adjacent $t-1$-key sibling, then descend into the merged node.

After deletion, if the root has zero keys and one child, replace the root by that child; this is the only way height decreases.

Theorem 10.11 (B-tree deletion correctness and complexity)**.** *The algorithm removes exactly k when present, preserves every B-tree invariant, and costs $\mathcal{O}(th)$ RAM time and $\mathcal{O}(h)$ page transfers.*

Proof. The cases cover whether the key is in the current node and whether the node is a leaf. Replacing an internal separator by its predecessor or successor preserves all key-range inequalities; recursive deletion removes the duplicate occurrence.

Borrowing preserves total keys and order: a parent separator enters the deficient child on the adjacent side, and the sibling's extreme key becomes the new separator. Both children then have at least $t-1$ keys. Merging two $t-1$-key siblings plus one separator creates exactly $2t-1$ keys, within the maximum, while the parent loses one key and child. The top-down precondition says the parent being descended from was nondeficient, except possibly the root, so no illegal nonroot underflow is created.

Consequently every recursive target child begins with at least t keys and can lose one safely. Equal leaf depth is unchanged by local borrow/merge; contracting an empty root removes one level from every leaf simultaneously. Search order and the transformations show exactly the target is deleted. One root-to-leaf path is followed and each level moves $\mathcal{O}(t)$ keys/pointers in a constant number of nodes. □

Deletion code is case-heavy because it maintains minimum occupancy proactively. A correct implementation should use explicit helper procedures for borrow-left, borrow-right, and merge and test every boundary index.

10.8 Red-black trees

EXAMINATION SYLLABUS: ITEM 76

Use explicit or shared sentinel leaves `NIL`; sentinels are black and contain no dictionary key.

Definition 10.12 (Red-black tree)**.** A red-black tree is a BST with a color on every internal node such that:

1. every node is red or black;
2. the root is black;
3. every `NIL` leaf is black;
4. a red node has black children;
5. for each node v, every path from v to a descendant `NIL` contains the same number of black nodes.

The common number of black internal nodes below v, plus the sentinel according to a consistent convention, is the **black height** $bh(v)$. Rotations preserve BST order but colors must be changed so black heights remain equal.

10.9 Red-black height

EXAMINATION SYLLABUS: ITEM 77

Lemma 10.13. *A subtree rooted at a node of black height b contains at least $2^b - 1$ internal nodes.*

Proof. Induct on b. For $b = 0$, the bound is zero. Each child has black height at least $b - 1$: it is $b - 1$ if the child root is black and b if red under the standard convention. By induction, the two child subtrees contain at least $2^{b-1} - 1$ nodes each; including the root gives $2^b - 1$. □

Theorem 10.14 (Red-black depth bound)**.** *A red-black tree with n internal nodes has height*

$$h \le 2 \log_2(n + 1).$$

Proof. No two red nodes are adjacent, so at least half the internal nodes on any root-to- leaf path are black. Thus $h \le 2bh(root)$. The lemma gives $n \ge 2^{bh(root)} - 1$, so $bh(root) \le \log_2(n + 1)$. Combine the inequalities. □

Consequently ordinary BST search is $\mathcal{O}(\log n)$ worst case.

10.10 Red-black insertion

EXAMINATION SYLLABUS: ITEM 78

Insert z as an ordinary BST leaf and color it red. Its black `NIL` children and every black height remain unchanged; the only possible violations are a red root or a red parent with red child. The following is the standard fix-up; the other orientation is its mirror image.

Listing 10.4: Red-black insertion fix-up

```
procedure InsertFix(z):
    while z.parent.color == RED:
        if z.parent == z.parent.parent.left:
            y = z.parent.parent.right          // uncle
            if y.color == RED:                 // Case 1
                z.parent.color = BLACK
                y.color = BLACK
                z.parent.parent.color = RED
                z = z.parent.parent
            else:
                if z == z.parent.right:        // Case 2
                    z = z.parent
                    RotateLeft(z)
                z.parent.color = BLACK         // Case 3
                z.parent.parent.color = RED
                RotateRight(z.parent.parent)
        else:
            perform the exact left/right mirror of the above cases
    root.color = BLACK
```

Theorem 10.15 (Insertion fix-up correctness)**.** *The algorithm restores all red-black properties in* $\mathcal{O}(\log n)$ *worst-case time, using at most two rotations.*

Proof. Maintain that the only possible nonroot violation is the red-red edge from z to its parent, and black heights are otherwise equal.

In Case 1, parent and uncle are red. Recoloring them black and grandparent red preserves the black count on every path through the grandparent: each gains one at the child and loses one at the grandparent. It removes the local red-red edges but may create one between the grandparent and its parent, so the violation moves two levels upward.

With a black uncle, Case 2 rotates a triangular z-parent configuration into the outer configuration without changing colors or path black counts. In Case 3, coloring the parent black and grandparent red, then rotating, makes the black parent the local root with two red children-side roots; no red-red edge remains and every path keeps the same number of black nodes. The symmetric cases have identical logic.

Each Case 1 iteration moves z upward two levels. Cases 2–3 terminate after at most two rotations. Finally coloring the root black removes a root violation and adds one black node equally to every root-to-leaf path. Height is logarithmic. □

Parent pointers, the global root, and any subtree aggregates must be updated by the rotation primitives. The official case sheet changes bookkeeping, not the proof invariant.

10.11 Red-black deletion

EXAMINATION SYLLABUS: ITEM 79

Perform ordinary BST deletion, physically removing a node y with at most one non-sentinel child x (use the successor when the requested node has two children). If y was red, all black heights remain valid. If y was black, regard x as carrying one extra unit of blackness. The fix-up moves or removes this **double black**. Sentinels need valid parent pointers during repair.

Listing 10.5: Red-black deletion fix-up

```
procedure DeleteFix(x):
    while x != root and x.color == BLACK:
        if x == x.parent.left:
            w = x.parent.right                 // sibling
            if w.color == RED:                 // Case 1
                w.color = BLACK
                x.parent.color = RED
                RotateLeft(x.parent)
                w = x.parent.right
            if w.left.color == BLACK and
               w.right.color == BLACK:         // Case 2
                w.color = RED
                x = x.parent
            else:
                if w.right.color == BLACK:     // Case 3
                    w.left.color = BLACK
```

```
                    w.color = RED
                    RotateRight(w)
                    w = x.parent.right
                w.color = x.parent.color       // Case 4
                x.parent.color = BLACK
                w.right.color = BLACK
                RotateLeft(x.parent)
                x = root
        else:
            perform the exact left/right mirror of the above cases
    x.color = BLACK
```

Theorem 10.16 (Deletion fix-up correctness)**.** *After deletion and fix-up, all red-black properties hold. The operation takes* $\mathcal{O}(\log n)$ *worst-case time and at most three rotations.*

Proof. The BST splice preserves key order. Model the removed black node by an extra black on x; then all paths still have equal *effective* black count, and the loop must eliminate or move this extra unit.

Assume x is a left child; mirror cases are symmetric.

Case 1: red sibling.
Its parent and children are black. Recoloring sibling black and parent red, then rotating left, preserves effective black counts and converts the configuration to one with a black sibling. It is a normalization step.

Case 2: black sibling with two black children.
Recoloring the sibling red removes one ordinary black from the sibling paths; combining that deficit with x's extra black makes the two sides equal, but the parent now carries the extra black. Set x to the parent and continue.

Case 3: black sibling, near child red, far child black.
Recolor the near child black and sibling red, then rotate right at the sibling. Black counts are preserved and the new sibling has a red far child, producing Case 4.

Case 4: black sibling with red far child.
Give the sibling the parent's color, make parent and far child black, and rotate left at the parent. Every path through the repaired subtree receives exactly the required ordinary black count; the extra black is absorbed, so setting $x = root$ terminates.

If the loop reaches a red x, coloring it black absorbs the extra unit. If it reaches the root, the extra black may be discarded because every root-to-leaf path contains it equally. Thus all black heights and red-parent rules are restored. Case 2 moves upward one level, so there are $\mathcal{O}(h) = \mathcal{O}(\log n)$ iterations; Cases 1, 3, and 4 contribute at most three rotations total. □

The most common implementation errors are stopping search at a sentinel whose parent was not set, forgetting to save the physically removed node's original color, and confusing the near and far nephews after a rotation.

10.12 Comparing search-tree implementations

EXAMINATION SYLLABUS: ITEM 80

No search tree dominates every workload. The bounds below assume n stored keys and count ordered-dictionary operations.

Structure	Search/update	Balance metadata	Merge/split	Principal use and tradeoff
Naive BST	$\mathcal{O}(h)$, worst $\Theta(n)$	none	$\mathcal{O}(h)$ variants	Small or already-random data; simplest, but input order can form a chain.
AVL	worst $\Theta(\log n)$	height or balance factor	possible but not its simplest strength	Tighter height and fast searches; more rotations/bookkeeping on updates.
Splay	amortized $\mathcal{O}(\log n)$, one operation $\Theta(n)$	none; aggregates optional	amortized $\mathcal{O}(\log n)$	Adapts to locality; elegant sequences and splits; no latency guarantee.
B-tree	worst $\Theta(\log_t n)$ node I/Os	key count and arrays	specialized bulk operations	High fan-out minimizes disk/cache transfers; complex deletion and node movement.
Treap	expected $\mathcal{O}(\log n)$, worst $\Theta(n)$	random priority	expected $\mathcal{O}(\log n)$	Very simple merge/split and implicit sequences; needs trustworthy randomness.
Red-black	worst $\Theta(\log n)$	one color bit	possible but less direct than treap	Moderate balance, bounded updates, standard library maps; deletion cases are intricate.

Height versus updates. AVL trees enforce balance factor at every node and have a smaller worst-case height constant than red-black trees. Red-black trees allow more shape flexibility and bound insertion/deletion rotations tightly, often favoring update-heavy general dictionaries.

Worst-case versus adaptive/expected guarantees. AVL and red-black trees give per-operation worst-case logarithmic time. Splay trees give deterministic amortized time and exploit locality. Treaps give simple expected bounds over priorities. These guarantees are logically different and must not be reported interchangeably.

Memory hierarchy. Pointer-based binary trees may incur a cache miss at every level. B-trees trade more in-node comparisons and shifts for far fewer expensive page transfers. For small in-memory sets, a sorted vector can outperform all trees despite linear insertion because of compactness and locality.

Augmentation. Any rotation-based BST can store subtree size, sum, minimum, or another associative aggregate if every mutation recomputes affected nodes bottom-up. Treaps and splay trees are especially convenient when split/merge expresses the desired range operation; AVL and red-black trees are preferable when strict worst-case dictionary latency is the primary contract.

LECTURE UNIT A

Syllabus Coverage Index

This appendix is a completeness certificate for the supplied examination programme. Every integer $1, \dots, 80$ occurs exactly once as a framed syllabus marker in the main text. Page references below lead to the corresponding marker; each marked section states the problem, algorithm or representation, correctness argument, and asymptotic analysis whenever applicable.

Item	Required topic	Page
1	RAM computation model, allowed operations, and measuring running time	2
2	$\mathcal{O}, \Omega, \Theta$; independence of starting index; examples	3
3	Static range sums by prefix sums	5
4	Membership in a sorted array by binary search	6
5	Sorting problem; two formulations and their equivalence	8
6	Proof that $\log(n!) = \Theta(n \log n)$	9
7	Comparison-sorting lower bound	9
8	Merge sort	10
9	Solving $T(n) = 2T(n/2) + \mathcal{O}(n)$	11
10	Counting array inversions	12
11	Quicksort, including expected asymptotics	13
12	Random-pivot quickselect, including expected asymptotics	16
13	Deterministic $\mathcal{O}(n)$ selection (median of medians)	17
14	Deterministic $\mathcal{O}(n \log n)$ quicksort	18
15	Stable counting sort and sorting pairs	19
16	LSD radix sort	20
17	Singly and doubly linked lists; insertion and deletion	23
18	Stack implemented by a list	24
19	Validation of multiple bracket types	25
20	Nearest smaller/greater element on either side	25
21	Evaluation of reverse Polish notation	26
22	Minimum-supporting stack	27
23	Queue implemented by a list	28
24	Queue on two stacks; amortized bounds and queue minimum	28
25	Binary heap definition, array layout, and heap property	30
26	`siftUp` and `siftDown` with correctness proofs	30
27	`insert`, `getMin`, `extractMin`, `decreaseKey`	32
28	Linear-time Floyd heap construction	32
29	In-place heapsort; impossibility of constant-time insert and extract-min	33
30	Heap deletion by pointer/handle	34
31	Heap deletion by value	35
32	Binomial trees and binomial heaps; advantage over binary heaps	35
33	Binomial-heap merge, insert, minimum, extraction, and decrease-key	36
34	Amortized analysis and accounting time	36
35	Accounting/coin method on a two-stack queue	37
36	Dynamic array implementation and amortized analysis	37
37	Sparse table: $\mathcal{O}(n \log n)$ build and $\mathcal{O}(1)$ query	38
38	Segment tree model problem, queries, and time proof	41

Item	Required topic	Page
39	Segment-tree binary descent and threshold-subarray search	42
40	Lazy segment-tree range assignment and `push`	43
41	Range count of values $\leq x$ by fractional cascading	44
42	Dynamic segment tree	45
43	Coordinate compression for segment trees	46
44	Persistent segment tree	47
45	Range count of values $\leq x$ by persistent segment tree	47
46	Range k-th statistic by parallel descent in two versions	48
47	Fenwick tree, `update`, and `getSum`	49
48	Higher-dimensional Fenwick trees and asymptotics	50
49	Reverse Fenwick maximum and monotone point update	50
50	Fenwick tree of Fenwick trees	52
51	BST definition, find/insert/erase, merge/split, order queries	54
52	AVL definition and depth bound	55
53	Repairing AVL imbalance	56
54	AVL find, insert, and erase	57
55	Splay-tree definition and practical significance	59
56	Zig, zig-zig, zig-zag, and splay	59
57	Potential method	60
58	Amortized splay bound by potential	61
59	Splay insert, erase, and find; relation to splay	62
60	Splay merge and split	63
61	Implicit-key tree: sequence updates and range sum	63
62	Hashing, collisions, universal and k-independent families	66
63	Chaining table operations and rehash	67
64	Perfect hashing	68
65	Open addressing, rehash, linear probing, and double hashing	69
66	Bloom filter and optimal parameters	71
67	Cuckoo filter	72
68	Skip-list find, insert, erase, and expected asymptotics	73
69	Cartesian tree, expected depth, and linear construction	76
70	Treap merge and split	77
71	Treap insert and erase via merge/split	78
72	B-tree definition, practical significance, and find	79
73	B-tree depth bound	79
74	B-tree insertion	80
75	B-tree deletion	81
76	Red-black tree definition	82
77	Red-black depth bound	83
78	Red-black insertion	83
79	Red-black deletion	84
80	Comparison of naive, AVL, splay, B-, treap, and red-black trees	86

LECTURE UNIT B

Reference Sheets and Further Reading

B.1 Operation bounds at a glance

The table records the principal guarantees proved in the text. "Expected" is over internal randomness; "amortized" is a deterministic sequence bound.

Structure/algorithm	Main time bound	Space or qualification
Binary search	$\Theta(\log n)$	sorted static array
Merge sort	$\Theta(n \log n)$ worst case	$\Theta(n)$ auxiliary, stable
Randomized quicksort	expected $\Theta(n \log n)$	worst $\Theta(n^2)$, in-place
BFPRT selection	$\Theta(n)$ worst case	comparison model
Counting sort	$\Theta(n + K)$	integer keys in $[0, K)$
Binary heap operation	$\mathcal{O}(\log n)$	minimum $\Theta(1)$
Heapify	$\Theta(n)$	in-place
Sparse-table RMQ	query $\Theta(1)$	build/space $\Theta(n \log n)$
Segment tree	$\Theta(\log n)$ per operation	$\Theta(n)$ space
Fenwick tree	$\Theta(\log n)$ per operation	$\Theta(n)$ space
Persistent segment update	$\Theta(\log n)$	same number of new nodes
AVL/red-black tree	$\Theta(\log n)$ worst case	per dictionary operation
Splay tree	$\mathcal{O}(\log n)$ amortized	one operation may be $\Theta(n)$
Treap	expected $\mathcal{O}(\log n)$	independent random priorities
B-tree	$\Theta(\log_t n)$ page depth	node capacity $\Theta(t)$
Chaining hash table	expected $\mathcal{O}(1 + \alpha)$	universal hashing
Open addressing	expected $\mathcal{O}(1)$ at fixed load	model/hash assumptions stated
FKS perfect hashing	lookup $\Theta(1)$ worst case	static; expected linear build
Skip list	expected $\mathcal{O}(\log n)$	worst $\Theta(n)$

B.2 Pseudocode conventions

- $a[l..r)$ denotes a half-open range. Empty ranges have $l = r$.
- An operation that reads `top`, `front`, or a tree root assumes nonemptiness unless an explicit error branch appears.
- `Update(v)` recomputes all stored subtree fields from v's own record and its children in constant time.
- `null` child aggregates use the operation's identity: size and sum are zero, minimum is $+\infty$, and maximum is $-\infty$.
- Random choices are uniform over the displayed finite set and mutually independent unless stated otherwise.

- Arithmetic affecting indices uses a type wide enough to avoid overflow; for example, use $l + (r - l)/2$, not $(l + r)/2$.

B.3 An oral-proof checklist

For any examination algorithm, check the following in order.

1. **Specification.** What is input, output, indexing convention, and every necessary assumption?
2. **Representation.** What does each stored field mean?
3. **Invariant.** Which statement is true before and after every loop, recursion, rotation, or lazy push?
4. **Preservation.** Why does every branch maintain it?
5. **Termination.** Which integer decreases or which interval shrinks?
6. **Postcondition.** Why does invariant plus termination give exactly the requested answer, including multiplicities and boundary cases?
7. **Time.** Count visited levels, pushes/pops, comparisons, or charged events; do not infer complexity from syntax alone.
8. **Memory.** Count arrays, allocated nodes, recursion depth, and persistence copies separately.

B.4 Selected references

These notes are self-contained for the stated syllabus. The following standard texts provide alternative proofs, historical context, and further exercises.

Bibliography

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[2] J. Erickson, *Algorithms*, self-published open textbook, 2019.

[3] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998.

[4] K. Mehlhorn and P. Sanders, *Algorithms and Data Structures: The Basic Toolbox*, Springer, 2008.

[5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.

[6] D. D. Sleator and R. E. Tarjan, "Self-adjusting binary search trees," *Journal of the ACM*, 32(3), 1985, pp. 652–686.

Closing Perspective

The central discipline of data-structure design is to make the invariant explicit, make every mutation preserve it, and charge every unit of running time to a mathematical object that cannot be charged too often. The algorithms in these notes differ in appearance, but their proofs repeatedly use this same pattern.